DOCUMENTATION WEEK 5

MONDAY

holiday: Pfingsten

TUESDAY

morning

- support with Yann & Stefanie
    - feedback:
        - How to create one piece from the single snippets: comment in & out; overlay functions; hardcut functions or fade
        - interaction between performers and code must be visible -> no hard-coding
        - inversion effect? try it with pGraphics
        - change visuals on beat
        - change colours of the visuals?

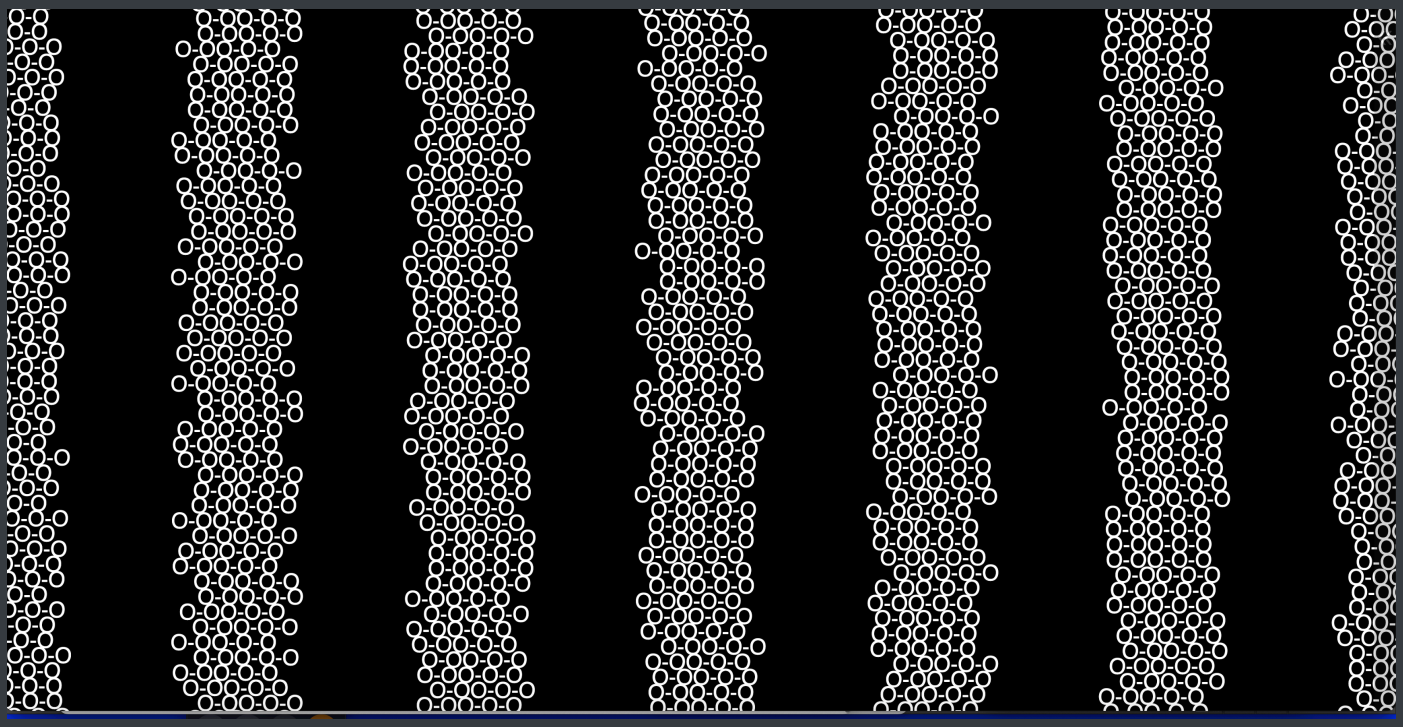
afternoon

- self-study: create further visuals to mix up in the performance

RANDOM OFFSET COLUMN WRITER

```
let wordCount = 6
let posX = 0
let posY = 0
let offsetX = 0
let word = "0-00-0-0"
```

```
let layer

function setup() {
  createCanvas(windowWidth, windowHeight)
  layer = createGraphics (windowWidth, windowHeight)
}

function draw() {
  //background (0)

  layer.clear()
  layer.fill(255)
  layer.textAlign(CENTER)
  layer.textSize(30)
  let k = frameCount%9

for(let i = 0; i <= wordCount; i++) {
    posX = map(i, 0, wordCount, 0, width)
    offsetX = 2* random(-10,10)
    posX = posX + offsetX
    layer.text(word, posX, posY)
    posY = posY + layer.textSize()/10
    if (k===1) {
      fill(0,255,0)
      posX = map(i, 0, wordCount, 0, width)
    }
  }

    image(layer, 0, 0)

//print (offsetX)
}
```

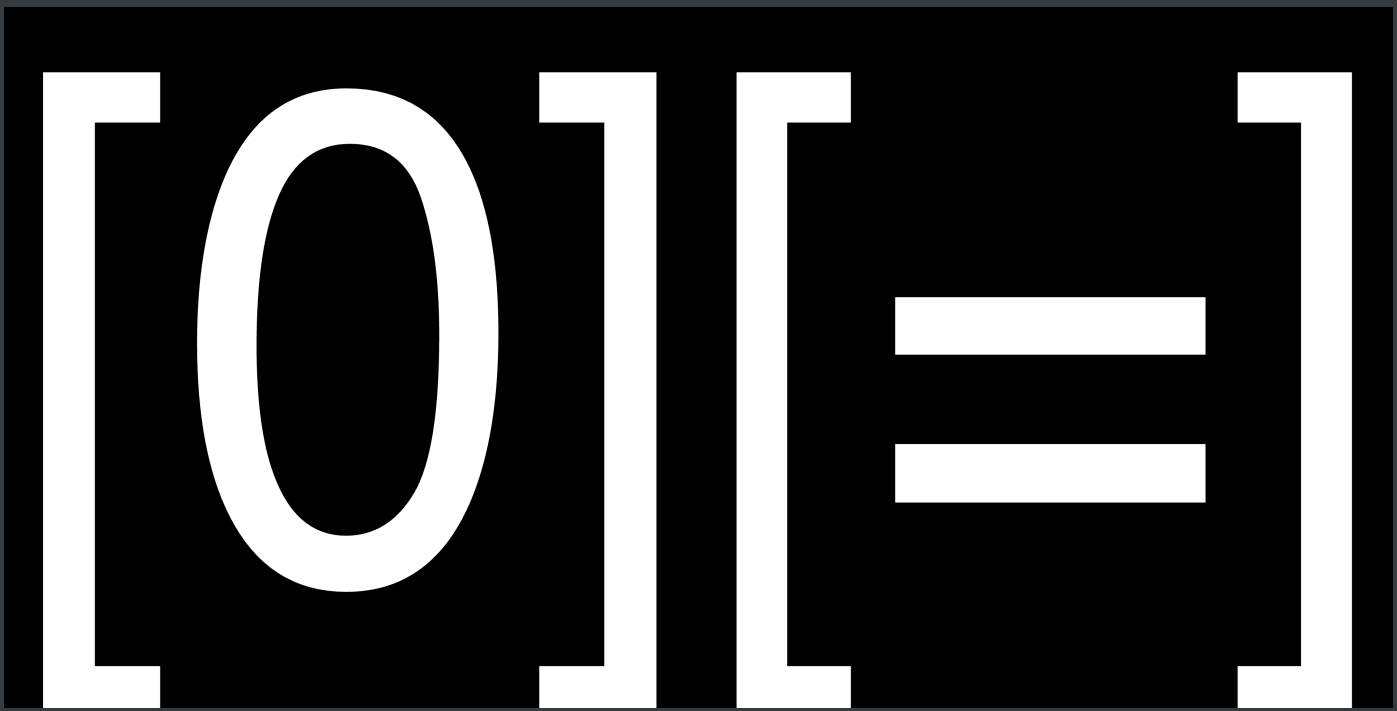


STRETCHED LETTER 4X4

```
let gridY = 2
let gridX = 2
let letter1 = "[0]"
let letter2 = "[=]"
let letter3 = "[%]"
let letter4 = "[+]"



function setup() {
  createCanvas(windowWidth, windowHeight)
  frameRate(3)

}

function draw() {
  background(0)
  fill(255)
  let columnWidth = width / gridX
  let lineHeight = height / gridY
```

```
let letterX1 = [1, 1, 1, 2, 1, 2]
let letterY1 = [2, 1, 2, 1, 1, 2]

let letterX2 = [1, 1, 1, 0, 1, 0]
let letterY2 = [1, 1, 2, 0, 2, 0]

let letterX3 = [1, 0, 0, 1, 0, 0]
let letterY3 = [1, 0, 0, 1, 0, 0]

let letterX4 = [0, 2, 0, 1, 1, 0]
let letterY4 = [0, 1, 0, 1, 1, 0]

textAlign(LEFT, TOP)


let i = frameCount % letterX1.length

//letter1
// calculates width of grid cells that should be filled with text
push()
let letterWidth1 = letterX1[i] * columnWidth
let letterHeight1 = letterY1[i] * lineHeight
textSize(letterHeight1)
let scaleX1 = letterWidth1 / textWidth(letter1)
scale(scaleX1, 1)
text(letter1, 0, 0)
pop()

// letter 2
push()
let letterWidth2 = letterX2[i] * columnWidth
let letterHeight2 = letterY2[i] * lineHeight
textSize(letterHeight2)
let scaleX2 = letterWidth2 / textWidth(letter2)
scale(scaleX2, 1)
text(letter2, columnWidth / scaleX2, 0)
pop()


// letter 3
```

```
    push()
    let letterWidth3 = letterX3[i] * columnWidth
    let letterHeight3 = letterY3[i] * lineHeight
    textSize(letterHeight3)
    let scaleX3 = letterWidth3 / textWidth(letter3)
    scale(scaleX3, 1)
    text(letter3, columnWidth / scaleX3, lineHeight)
    pop()

    // letter 4
    push()
    let letterWidth4 = letterX4[i] * columnWidth
    let letterHeight4 = letterY4[i] * lineHeight
    textSize(letterHeight4)
    let scaleX4 = letterWidth4 / textWidth(letter4)
    scale(scaleX4, 1)
    text(letter4, 0, lineHeight)
    pop()

  }
```

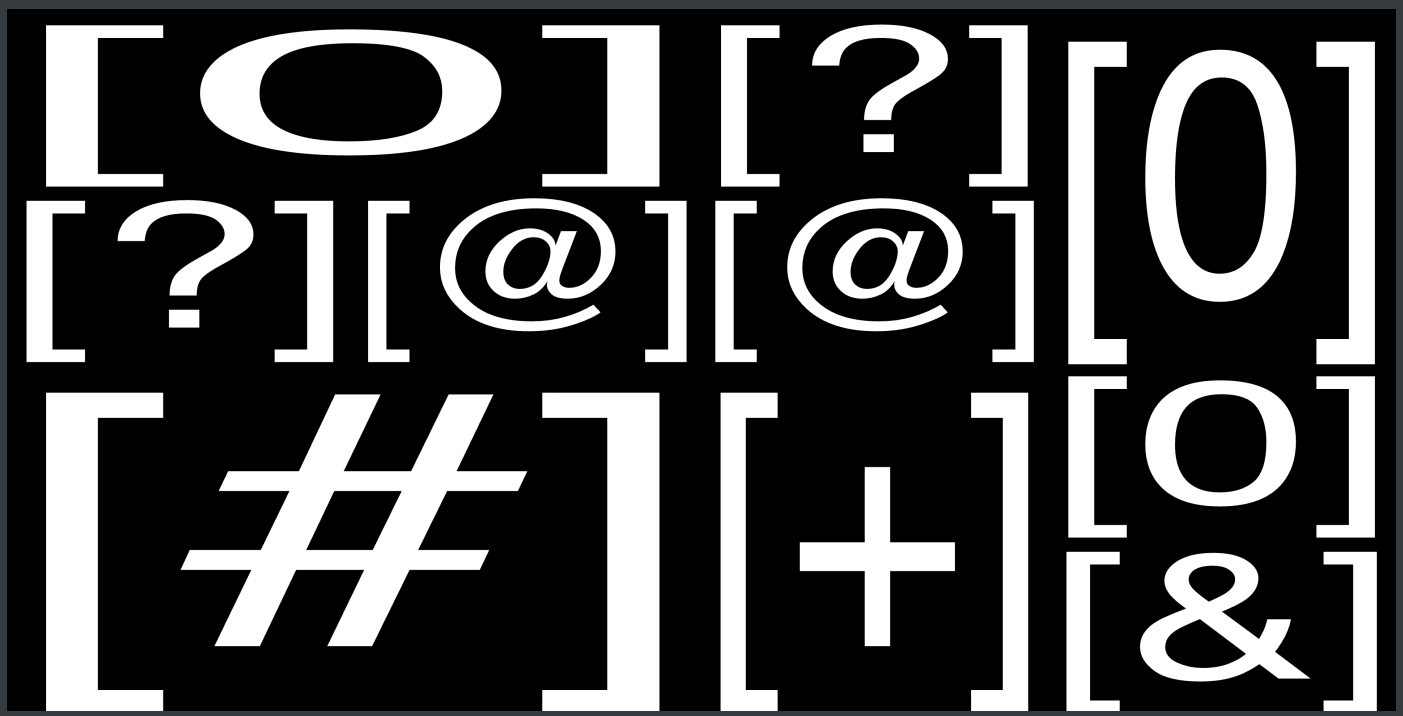


MULTIPLE STRETCHED LETTER 4X4 – FUNCTION

```
function setup() {
  createCanvas(windowWidth, windowHeight)
  frameRate(2)

}

function draw() {
background(0)
letter4x4("[0]","[=]","[@]","[?]", windowWidth/2, windowHeight/2, 0, 0, 0)
letter4x4("[?]","[0]","[!]","[@]", windowWidth/2, windowHeight/2, windowWidth/2,
0, 1)
letter4x4("[#]","[&]","[0]","[!]", windowWidth/2, windowHeight/2, 0,
windowHeight/2, 2)
letter4x4("[+]","[0]","[&]","[%]", windowWidth/2, windowHeight/2, windowWidth/2,
windowHeight/2, 3)









}


function letter4x4 (letter1, letter2, letter3, letter4, widthP, heightP, x, y,
offset) {
  let gridY = 2
  let gridX = 2
  fill(255)
  let columnWidth = widthP / gridX
  let lineHeight = heightP / gridY

  let letterX1 = [1, 1, 1, 2, 1, 2]
  let letterY1 = [2, 1, 2, 1, 1, 2]

  let letterX2 = [1, 1, 1, 0, 1, 0]
  let letterY2 = [1, 1, 2, 0, 2, 0]

  let letterX3 = [1, 0, 0, 1, 0, 0]
```

```
let letterY3 = [1, 0, 0, 1, 0, 0]

let letterX4 = [0, 2, 0, 1, 1, 0]
let letterY4 = [0, 1, 0, 1, 1, 0]

textAlign(LEFT, TOP)


let i = (frameCount + offset) % letterX1.length

push()
  translate(x, y)

//letter1
// calculates width of grid cells that should be filled with text
push()
let letterWidth1 = letterX1[i] * columnWidth
let letterHeight1 = letterY1[i] * lineHeight
textSize(letterHeight1)
let scaleX1 = letterWidth1 / textWidth(letter1)
scale(scaleX1, 1)
text(letter1, 0, 0)
pop()

// letter 2
push()
let letterWidth2 = letterX2[i] * columnWidth
let letterHeight2 = letterY2[i] * lineHeight
textSize(letterHeight2)
let scaleX2 = letterWidth2 / textWidth(letter2)
scale(scaleX2, 1)
text(letter2, columnWidth / scaleX2, 0)
pop()


// letter 3
push()
let letterWidth3 = letterX3[i] * columnWidth
let letterHeight3 = letterY3[i] * lineHeight
textSize(letterHeight3)
```

```
    let scaleX3 = letterWidth3 / textWidth(letter3)
    scale(scaleX3, 1)
    text(letter3, columnWidth / scaleX3, lineHeight)
    pop()

    // letter 4
    push()
    let letterWidth4 = letterX4[i] * columnWidth
    let letterHeight4 = letterY4[i] * lineHeight
    textSize(letterHeight4)
    let scaleX4 = letterWidth4 / textWidth(letter4)
    scale(scaleX4, 1)
    text(letter4, 0, lineHeight)
    pop()

    pop()
}
```



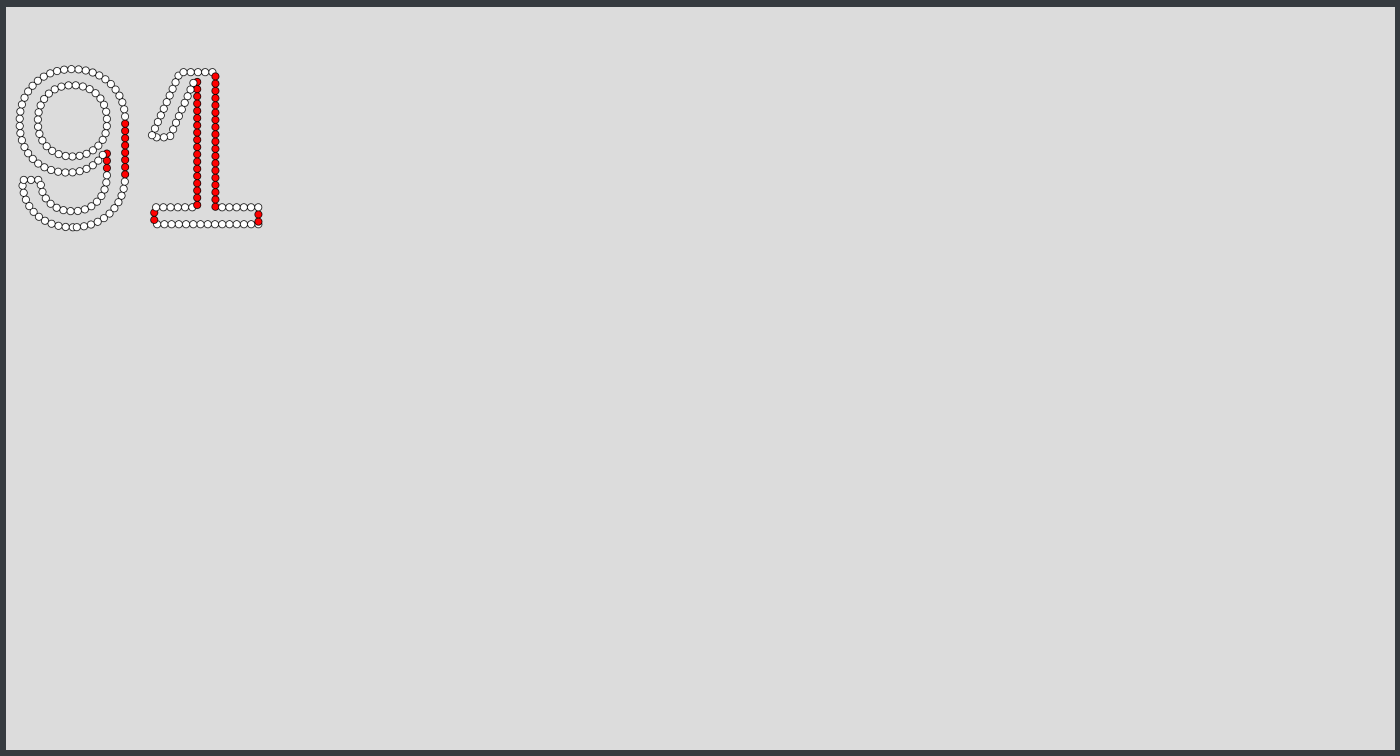
WEDNESDAY

morning

self-study

TUTORIAL TEXT-TO-POINTS

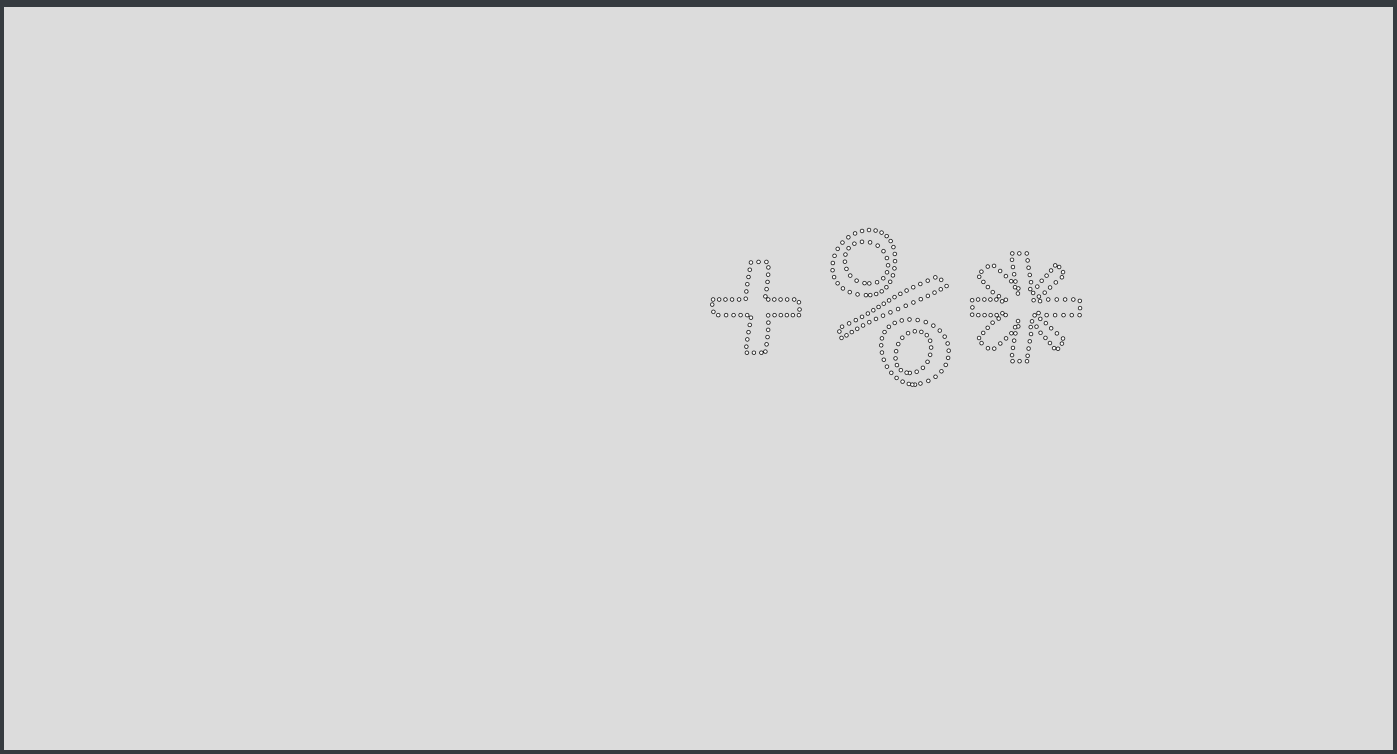
```
let points = []
let font

function preload() {
  font = loadFont('/data/spacemono.ttf')
}


function setup() {
  createCanvas(windowWidth, windowHeight)
  points = font.textToPoints("91", 0, 300, 300)
}

function draw() {

  background(220)
  for(let i = 0; i < points.length; i++) {

    //alpha stores the angle of the line at each point
    if (points[i].alpha==90) {
      fill(255,0,0)
    }
    else {
      fill(255)
    }

    ellipse(points[i].x, points[i].y, 10)
  }
}
```



TUTORIAL KINETIC TYPOGRAPHY

```
let points = []
let font

function preload() {
  font = loadFont('/data/spacemono.ttf')
}


function setup() {
  createCanvas(windowWidth, windowHeight)
  points = font.textToPoints("91", 0, 300, 300)
}

function draw() {

  background(220)
  for(let i = 0; i < points.length; i++) {

    //alpha stores the angle of the line at each point
    if (points[i].alpha==90) {
      fill(255,0,0)
```

```
    }
    else {
      fill(255)
    }

    ellipse(points[i].x, points[i].y, 10)
  }
}
```



TEXT-TO-POINT DISTORTED LETTER

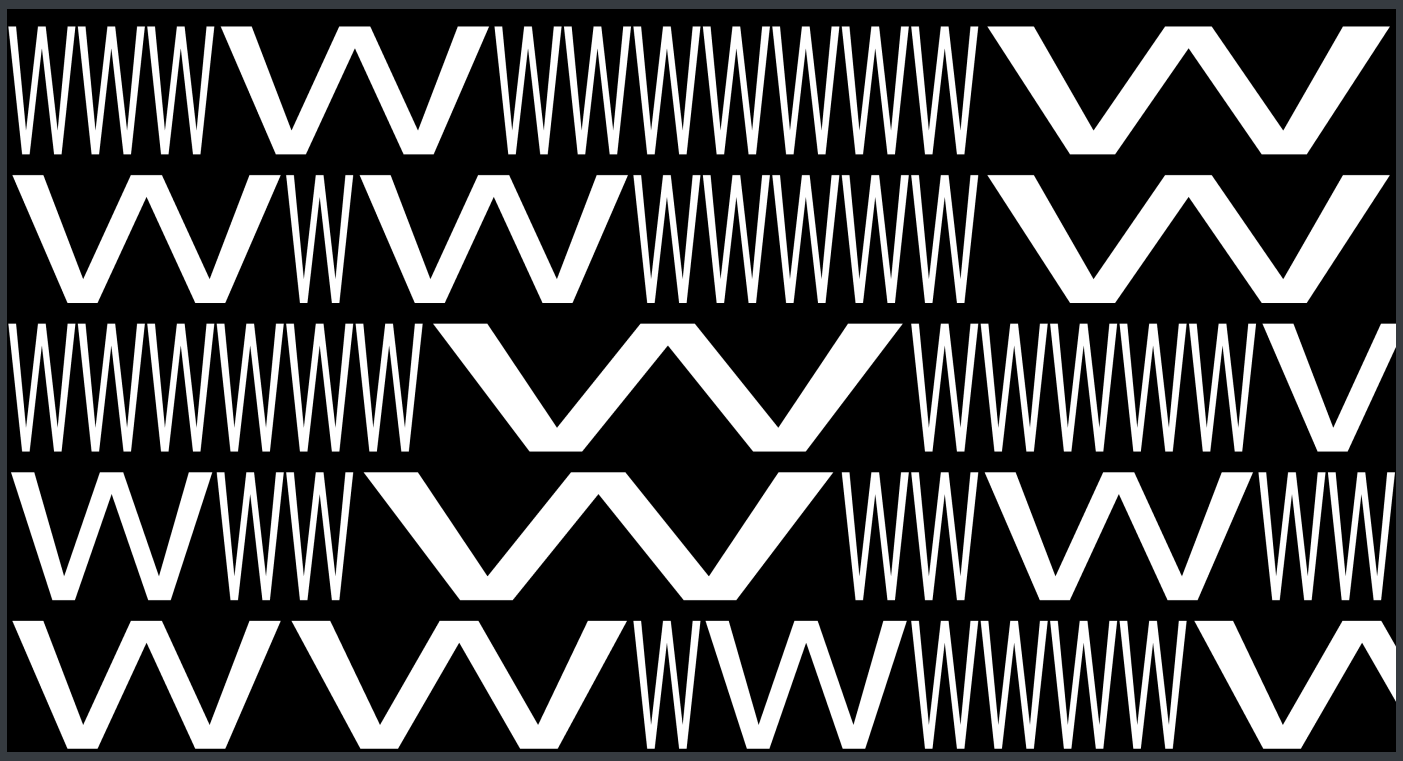
```
let points = []
let font

function preload() {
  font = loadFont('/data/spacemono.ttf')
}


function setup() {
  createCanvas(windowWidth, windowHeight)
```

```
    points = font.textToPoints("91", 0, 300, 300)
  }

  function draw() {

    background(220)
    for(let i = 0; i < points.length; i++) {

      //alpha stores the angle of the line at each point
      if (points[i].alpha==90) {
        fill(255,0,0)
      }
      else {
        fill(255)
      }

      ellipse(points[i].x, points[i].y, 10)
    }
  }
```

HORIZONTALLY STRETCHED LETTER ANIMATION

```
  let gridX = 20
```

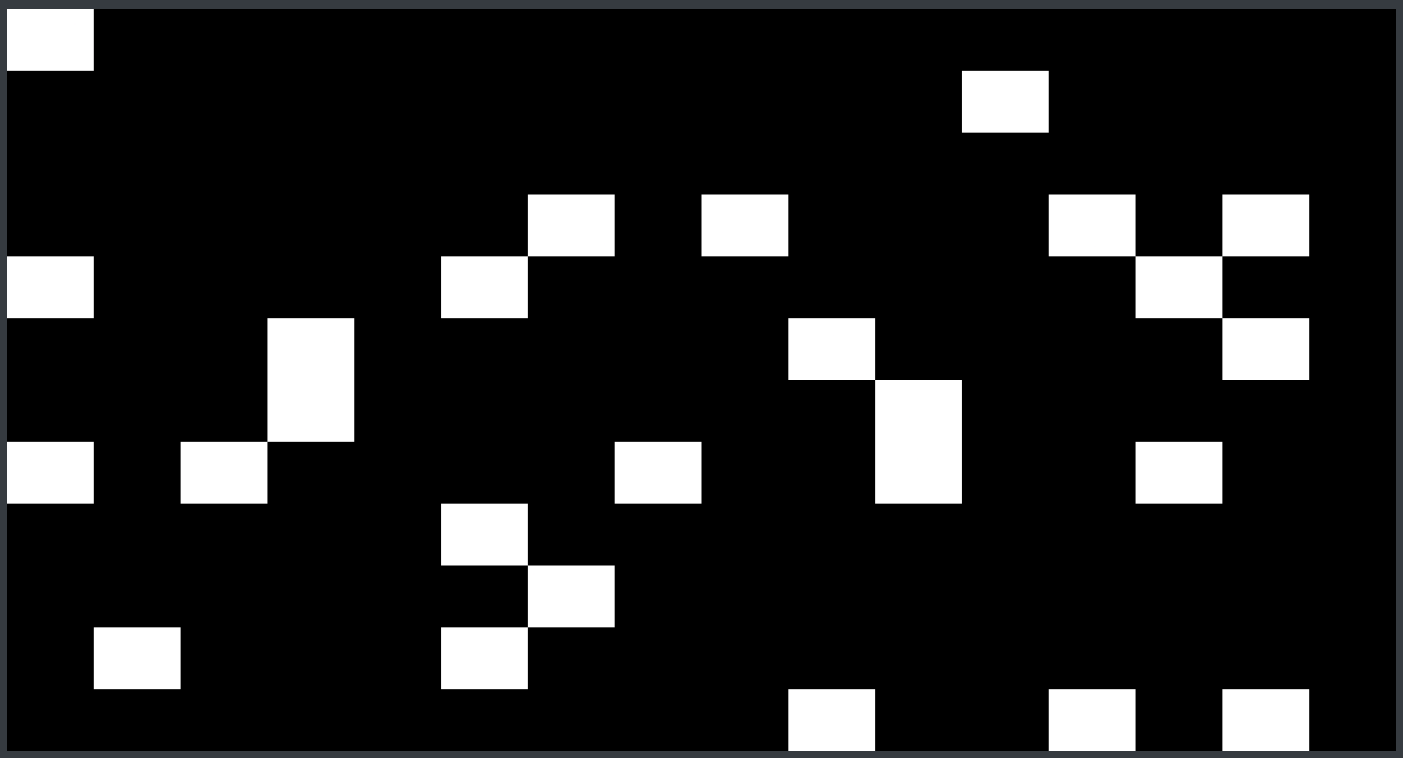
```
let gridY = 5
let letter = "W"

function setup() {
  createCanvas(windowWidth, windowHeight)
  frameRate(3)
}

function draw() {
  background(0)
  fill(255)
  textAlign(LEFT, TOP)




  let gridWidth = width / gridX
  let gridHeight = height / gridY
  textSize(gridHeight*1.2)
  let y = 0

  for(let k = 0; k < gridY; k++) {
    let x = 0
    for(let i = 0; i < gridX; i++) {
      //
      let letterX = random(1) < 0.2 ? floor(random(3, 8)) : 1
      let letterWidth = letterX * gridWidth
      let scaleX = letterWidth / textWidth(letter)
      push()
      translate(x, y)
      scale(scaleX, 1)
      text(letter, 0, k*gridHeight)
      pop()
      x += letterWidth
    }
  }
}
```



RANDOM RECTANGLE ANIMATION

```
function setup() {
  createCanvas(windowWidth, windowHeight)
  frameRate(3)
}

function draw() {
  background(0)
  noStroke()
  let numberX = 8
  let numberY = 4
  for(let k = 0; k < numberY + 1; k++) {
    for(let i = 0; i < numberX + 1; i++) {
      shape4x4(i * width / numberX, k * height/numberY, width / numberX, height
/ 2)
    }
  }
}

function shape4x4(x, y, width4, height4) {
```

```
  let gridY = 6
  let gridX = 2
  let columnWidth = width4 / gridX
  let lineHeight = height4 / gridY

  let shapeX1 = [1, 1, 1, 2, 1, 2]
  let shapeY1 = [2, 1, 2, 1, 1, 2]

  rectMode(LEFT, TOP)


  let i = frameCount % shapeX1.length

  //calculates how many gridCells are left
  let maxPosX = gridX - shapeX1[i]
  let maxPosY = gridY - shapeY1[i]

  let randomPosX = floor(random(0, max(1, maxPosX + 1)))
  let randomPosY = floor(random(0, max(1, maxPosY + 1)))
  let shapeWidth1 = shapeX1[i] * columnWidth
  let shapeHeight1 = shapeY1[i] * lineHeight
  rect(x + randomPosX * columnWidth, y + randomPosY * lineHeight, shapeWidth1,
shapeHeight1)
}
```



THURSDAY

morning

- brainstorming together with Julian
  - conceptual idea: topic of feeling overwhelmed in our everyday life (many catastrophic events; expectation for self-optimization; fast-paced life through capitalism; sensory overload; constant availability & access through phones; etc.)
  - visual idea: creating overwhelming feeling by splitting the screen into three areas where different snippets run at the same time -> overwhelming. activate and deactivate visuals on different areas during performance; colour inversion (strobo effect); area-filling visuals; overlay of different snippets?
  - auditive idea: adding text vocalizing this overwhelming feeling; overlay of different text snippets (?); fast-paced techno music
- coding attempts for visual material

BOUNCING LETTER STRETCH

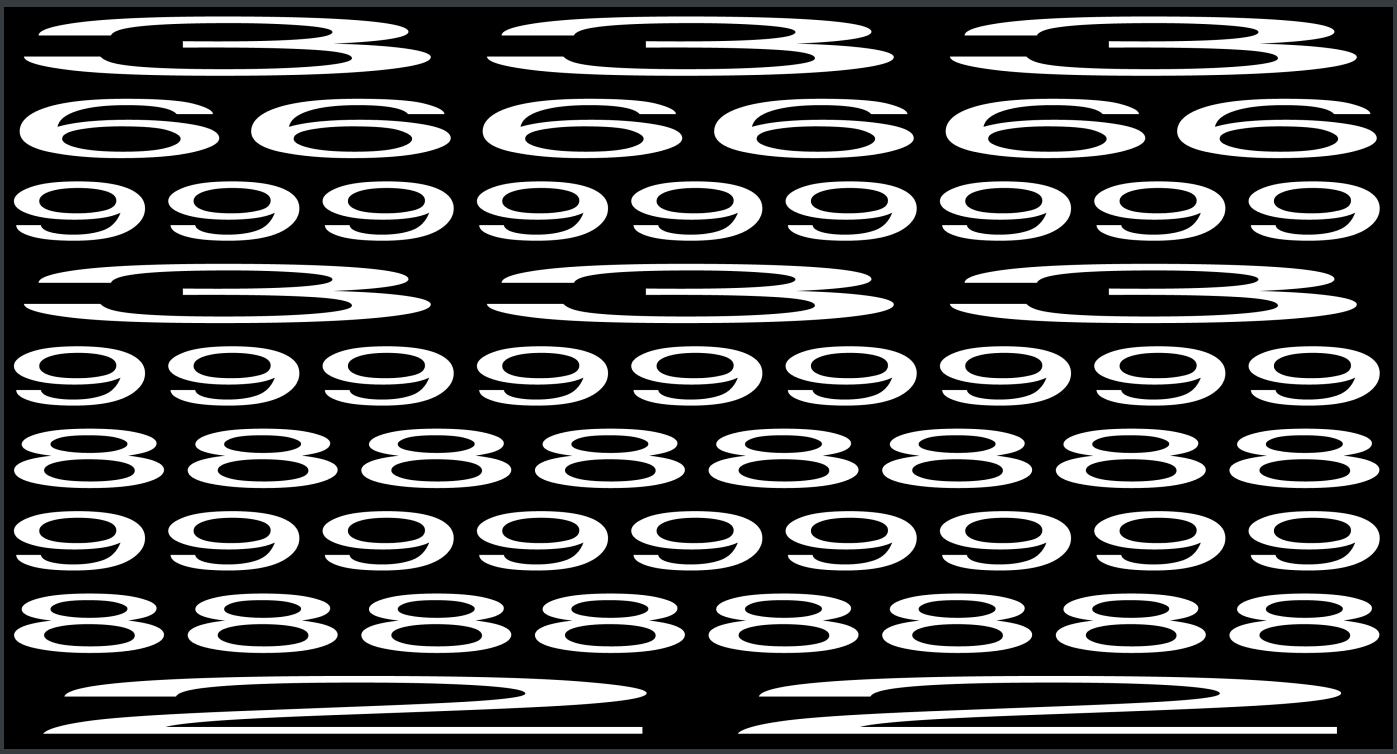
```
let letter = "&"
function setup() {
  createCanvas(windowWidth, windowHeight)

}

function draw() {
  background(0)
  fill(255)
  //stroke(0,255,255)


  let varySize = abs(sin(frameCount * 0.05)) * 2
  //let varySize = 1
  textSize(800 * varySize)
  textAlign(CENTER, CENTER)
```

```
  let scaleX = abs(sin(frameCount * 0.05)) * 6
  //let scaleX = 1

  push()
  scale(scaleX, 1)
  text(letter, 500, height/2)
  pop()


}
```

STRETCHED NUMBER COUNT

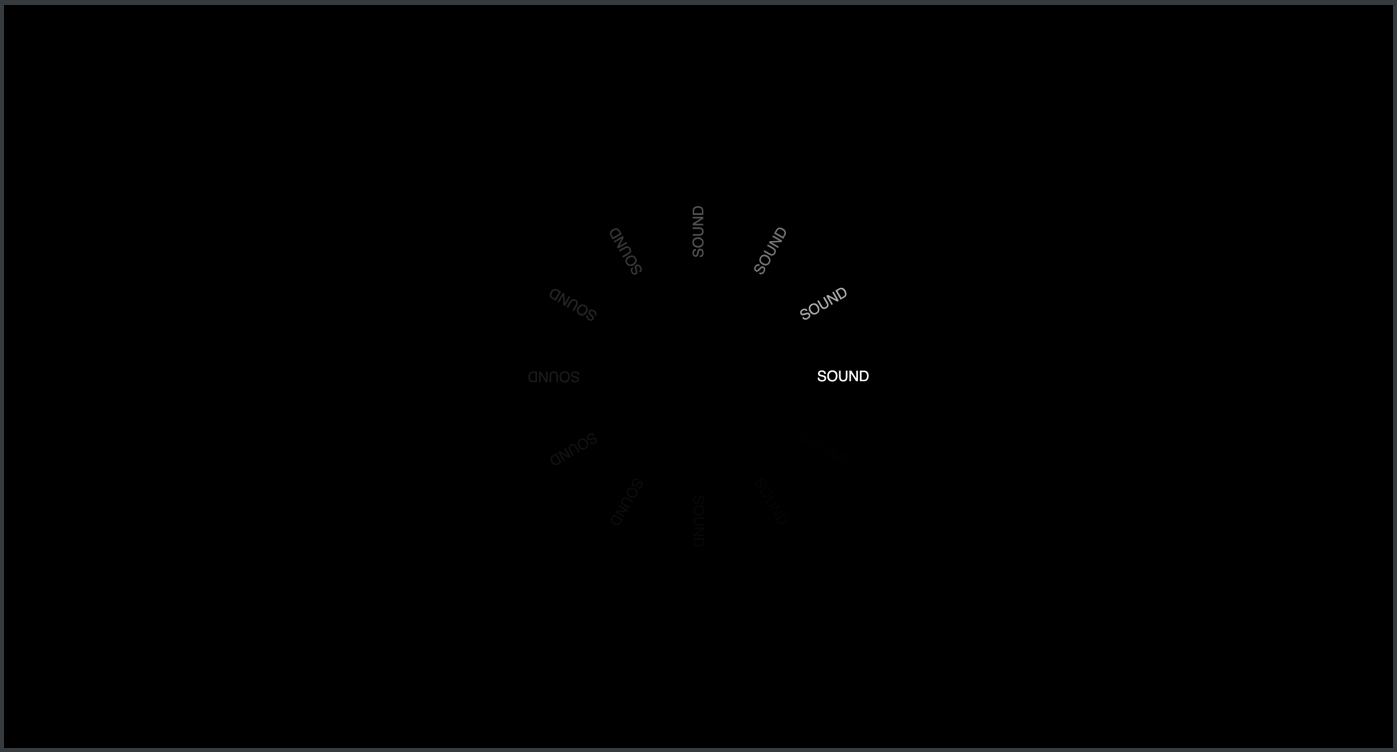
```
let amountX = 9

function setup() {
  createCanvas(windowWidth, windowHeight)
  frameRate(3)
```

```
}

function draw() {
  background(0)
  fill(255)
  textAlign(CENTER, TOP)
  let letter = ["1", "2", "3", "4", "5", "6", "7", "8", "9"]
  let letterNumber = [1, 2, 3, 4, 5, 6, 7, 8, 9]
  let letterHeight = height / amountX
  textSize(letterHeight)

  for(let k = 0; k < amountX; k++) {
  let i = (frameCount + k * floor(random(1,9))) % letterNumber.length
    let letterWidth = width / letterNumber[i]


    let scaleX = letterWidth / textWidth(letter[i])
    push()
    scale(scaleX, 1)
    text(letter[i].repeat(letterNumber[i]), (width / 2) / scaleX, k *
letterHeight)
    pop()
  }
}
```



WAITING – TEXT ANIMATION

```
let word = "SOUND"
let wordCount = 12


function setup() {
  createCanvas(windowWidth, windowHeight)
  frameRate(5)
}

function draw() {
  background(0, 80)
  fill(255)
  textSize(20)
  textAlign(CENTER, CENTER)
  let rotation = 360 / wordCount
  let posX = width / 2
  let posY = height / 2
  let radius = 200

  let i = frameCount % wordCount
  let angle = i * rotation

  let x = posX + cos(radians(angle)) * radius
  let y = posY + sin(radians(angle)) * radius
  textRot(word, x, y, angle)
}

function textRot(txt, x, y, angle) {
  push()
  translate(x, y)
  rotate(radians(angle))
  text(txt, 0, 0)
  pop()
}
```



afternoon

- experimenting further for visual material
- starting to transform snippets into functions

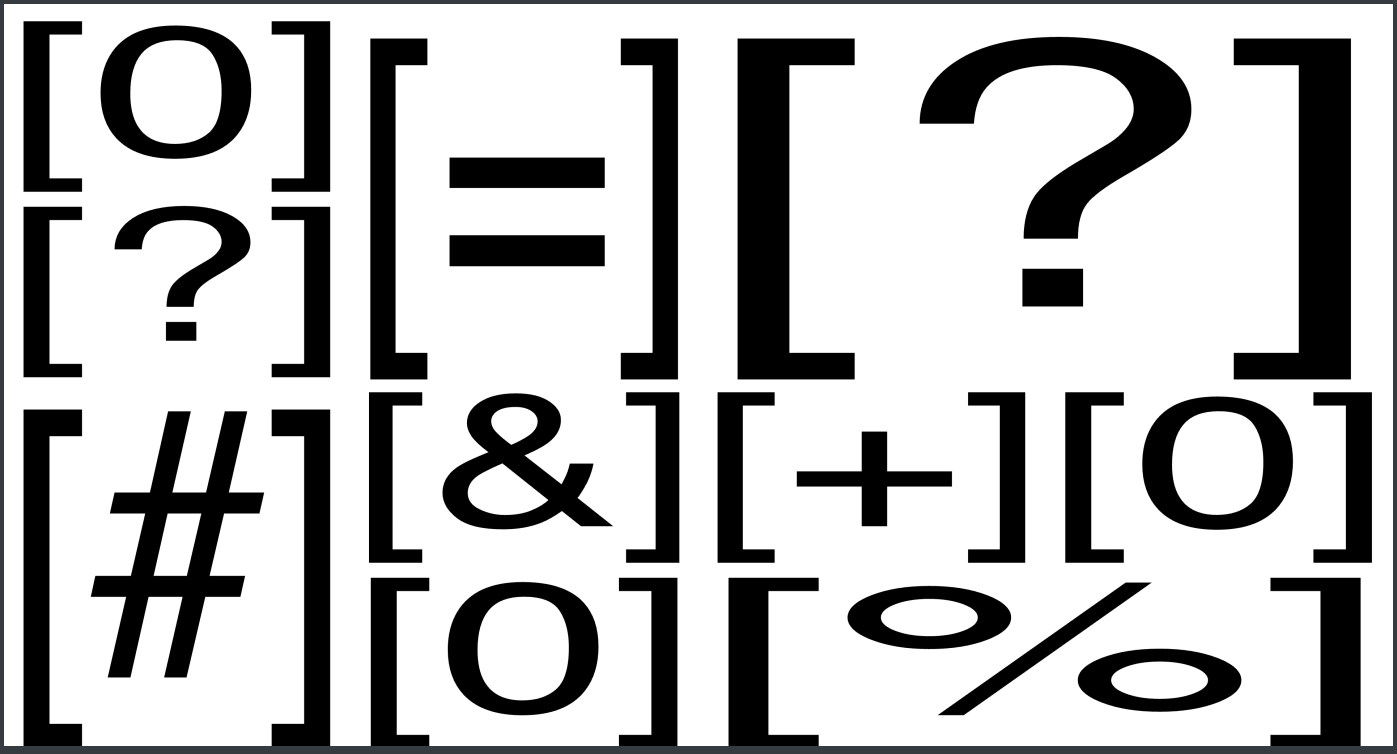
STROBO EFFECT TEST

```
function setup() {
  createCanvas(windowWidth, windowHeight)
  frameRate(3)

}

function draw() {
  let k = frameCount % 2

  if(k == 0) {
    fill(0)
    background(255)
  }
```

```
  else{
    fill(255)
    background(0)
  }


  letter4x4("[0]", "[=]", "[@]", "[?]", windowWidth / 2, windowHeight / 2, 0, 0,
0)
  letter4x4("[?]", "[0]", "[!]", "[@]", windowWidth / 2, windowHeight / 2,
windowWidth / 2, 0, 1)
  letter4x4("[#]", "[&]", "[0]", "[!]", windowWidth / 2, windowHeight / 2, 0,
windowHeight / 2, 2)
  letter4x4("[+]", "[0]", "[&]", "[%]", windowWidth / 2, windowHeight / 2,
windowWidth / 2, windowHeight / 2, 3)




}


function letter4x4(letter1, letter2, letter3, letter4, widthP, heightP, x, y,
offset) {
  let gridY = 2
  let gridX = 2

  let columnWidth = widthP / gridX
  let lineHeight = heightP / gridY

  let letterX1 = [1, 1, 1, 2, 1, 2]
  let letterY1 = [2, 1, 2, 1, 1, 2]

  let letterX2 = [1, 1, 1, 0, 1, 0]
  let letterY2 = [1, 1, 2, 0, 2, 0]

  let letterX3 = [1, 0, 0, 1, 0, 0]
  let letterY3 = [1, 0, 0, 1, 0, 0]

  let letterX4 = [0, 2, 0, 1, 1, 0]
  let letterY4 = [0, 1, 0, 1, 1, 0]
```

```
textAlign(LEFT, TOP)


let i = (frameCount + offset) % letterX1.length

push()
translate(x, y)

//letter1
// calculates width of grid cells that should be filled with text
push()
let letterWidth1 = letterX1[i] * columnWidth
let letterHeight1 = letterY1[i] * lineHeight
textSize(letterHeight1)
let scaleX1 = letterWidth1 / textWidth(letter1)
scale(scaleX1, 1)
text(letter1, 0, 0)
pop()

// letter 2
push()
let letterWidth2 = letterX2[i] * columnWidth
let letterHeight2 = letterY2[i] * lineHeight
textSize(letterHeight2)
let scaleX2 = letterWidth2 / textWidth(letter2)
scale(scaleX2, 1)
text(letter2, columnWidth / scaleX2, 0)
pop()


// letter 3
push()
let letterWidth3 = letterX3[i] * columnWidth
let letterHeight3 = letterY3[i] * lineHeight
textSize(letterHeight3)
let scaleX3 = letterWidth3 / textWidth(letter3)
scale(scaleX3, 1)
text(letter3, columnWidth / scaleX3, lineHeight)
pop()
```

```
  // letter 4
  push()
  let letterWidth4 = letterX4[i] * columnWidth
  let letterHeight4 = letterY4[i] * lineHeight
  textSize(letterHeight4)
  let scaleX4 = letterWidth4 / textWidth(letter4)
  scale(scaleX4, 1)
  text(letter4, 0, lineHeight)
  pop()

  pop()
}
```



COUNTING CLOCK ANIMATION

```
function setup() {
  createCanvas(windowWidth, windowHeight)
  frameRate(3)
}

function draw() {
  background(0)
```

```
  fill(255)
  count60(height / 2, width / 2, 0, 0)

  if(frameCount >= 20) {
    count60(height / 2, width / 2, width / 2, 0)
  }

  if(frameCount >= 40) {
    count60(height / 2, width/2, 0, height/2)
  }

    if(frameCount >= 60) {
    count60(height / 2, width/2, width/2, height/2)
  }

}


function count60(cellHeight, cellWidth, x, y) {
  textAlign(LEFT, TOP)
  let letter = ["00", "01", "02", "03", "04", "05", "06", "07", "08",
    "09", "10", "11", "12", "13", "14", "15", "16", "17", "18", "19",
    "20", "21", "22", "23", "24", "25", "26", "27", "28", "29", "30",
    "31", "32", "33", "34", "35", "36", "37", "38", "39", "40", "41",
    "42", "43", "44", "45", "46", "47", "48", "49", "50", "51", "52",
    "53", "54", "55", "56", "57", "58", "59", "60"
  ]

  textSize(cellHeight)


  let i = frameCount % letter.length
  let scaleX = cellWidth / textWidth(letter[i])
  push()
  scale(scaleX, 1)
  text(letter[i], x / scaleX, y)
  pop()

  if(i % 7 === 0) {
    fill(255)
```

```
      rectMode(CORNER)
      rect(x, y, cellWidth, cellHeight)
    }
  }
```

FRIDAY

- turning snippets into functions
- setting up the split screen for the performance and inserting functions

CURRENT STATE OF CODING CANVAS

```
// {"P5LIVE":{"name":"split_screen","mod":1780069414429}}

let leftLayer;
let centerLayer;
let rightLayer;

let blinkCount = 1
```

```
function setup() {
  createCanvas(windowWidth, windowHeight);
  leftLayer = createGraphics(width / 3, height);
  centerLayer = createGraphics(width / 3, height);
  rightLayer = createGraphics(width / 3, height);
  setupAudio(true) // global vars
  // a5.ease = .075 // set easing
}

function draw() {
  /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()
  let k = frameCount % blinkCount



  //LEFT LAYER
  leftLayer.background(0);
  leftLayer.fill(255)
  if(k < blinkCount / 2) {
    leftLayer.background(255)
    leftLayer.fill(0)
  }


  //

  // CENTER LAYER
  centerLayer.background(255);
  centerLayer.fill(0)

  if(k < blinkCount / 2) {
    centerLayer.background(0)
    centerLayer.fill(255)
  }

  //sinusTwist(centerLayer,width / 3, height, 0.3)
  //crossedTrigo(centerLayer, width / 3, height, 0.5, 0.1);
  //sinusString(centerLayer, width / 3, height, 0.1, 80);
  //sinusSpacing(centerLayer, width / 3, height, 0.2);
```

```
  //wordRotation(centerLayer, width / 3, height, 20, 8, 0.4);
  //sinusSpacingLeft(centerLayer, width / 3, height, 0.1, 50);
  //horizontalStretch(centerLayer, width/3, height, 10, 5, 18)
  //stretchedLetterGrid(centerLayer, width/3, height, 6, 8, 18)
  //randomShapeGrid(centerLayer, width / 3, height, 2, 8, 18);
  letter4x4(centerLayer, width/3, height, 1, 4, 18)

  //


  // RIGHT LAYER
  rightLayer.background(0);
  rightLayer.fill(255)

  if(k < blinkCount / 2) {
    rightLayer.background(255)
    rightLayer.fill(0)
  }

  //


  image(leftLayer, 0, 0);
  image(centerLayer, width / 3, 0);
  image(rightLayer, width / 3 * 2, 0);

}

//// NEW FUMNCTION /////
function crossedTrigo(g, w, h, bow, shape) {
  const elementY = 100;
  var charset = "000000000---------000000000";
  g.noStroke();
  g.textAlign(CENTER, CENTER);
  g.textSize(h / elementY);
  for(let row = 0; row < elementY + 1; row++) {
    for(let col = 0; col < charset.length + 1; col++) {
      let posY = map(row, 0, elementY, 0, h);
      let magX = map(tan(radians(posY * bow + frameCount * 0.5)), -1, 1, -w *
shape, w * shape);
```

```
      let posX = map(col, 0, charset.length, -magX, magX);
      g.push();
      g.translate(w / 2 + posX, posY);
      g.text(charset[col], 0, 0);
      g.pop();
    }
  }
}


//// NEW FUMNCTION /////
function sinusTwist(g, w, h, bow) {
  let word = "*/*##*/*";
  g.noStroke();
  g.textSize(20);
  g.textAlign(CENTER);

  for(let i = 0; i < 220; i++) {
    let varSpacing = abs(sin(frameCount * 0.03 + i * bow) * 10);

    let totalWidth = 0;
    for(let char of word) {
      totalWidth += g.textWidth(char) + varSpacing * 10;
    }
    totalWidth -= varSpacing * 10;
    let posX = w / 2 - totalWidth / 2;
    for(let char of word) {
      g.text(char, posX, 5 * i);
      posX += g.textWidth(char) + varSpacing * 10;
    }
  }
}


//// NEW FUMNCTION /////
function sinusString(g, w, h, bow, spread) {
  let word = "!%!%!%!%";
  let letters = word.split('');
  g.noStroke();
  g.textSize(20);
```

```
  for(let j = 0; j < 50; j++) {
    for(let i = 0; i < letters.length; i++) {
      let offsetX = sin(frameCount * 0.03 + j * bow) * spread * i;
      g.text(letters[i], w / 2 + offsetX * 0.5, j * 20);
    }
  }
}


//// NEW FUMNCTION /////
function sinusSpacing(g, w, h, bow) {
  let word = "[&*&]";
  let spread = 40;
  g.noStroke();
  g.textSize(20);
  g.textAlign(LEFT);

  for(let i = 0; i < 70; i++) {
    let spac = abs(sin(frameCount * 0.03 + i * bow) * spread);

    let totalWidth = 0;
    for(let char of word.repeat(8)) {
      totalWidth += g.textWidth(char) + spac;
    }
    totalWidth -= spac;
    let posX = w / 2 - totalWidth / 2;
    for(let char of word.repeat(30)) {
      g.text(char, posX, 20 * i);
      posX += g.textWidth(char) + spac;
    }
  }
}


//// NEW FUNCTION /////
function wordRotation(g, w, h, wordCount, rowCount, speed) {
    let word = "-#-%-#-";
    g.textSize(22);
    g.noStroke();
```

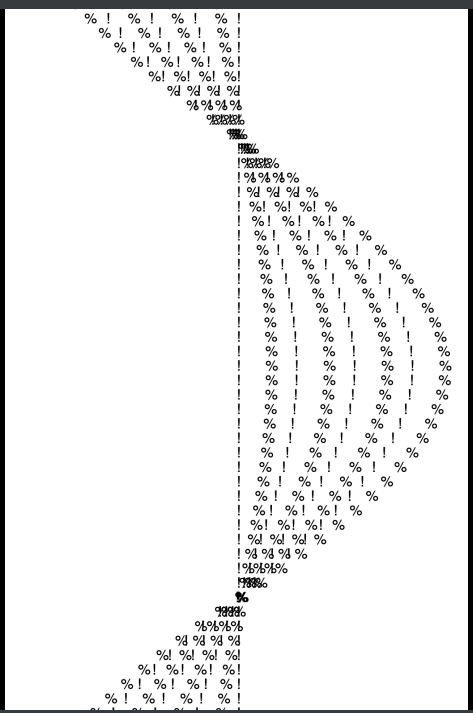
```
    let posX = w / 2;
    let posY = h / 2;
    let wordWidth = g.textWidth(word);
    for (let j = 0; j < rowCount; j++) {
        let radiusVar = sin(frameCount * 0.2) * 3 + 5 * ampEase;
        let radius = radiusVar + 5 + j * (wordWidth + 10);
        let direction = (j % 2 === 0) ? 1 : -1;
        let rotation = 360 / wordCount;
        for (let i = 0; i < wordCount; i++) {
            let rowOffset = frameCount * speed * direction;
            let angle = i * rotation + rowOffset;
            let x = posX + cos(radians(angle)) * radius;
            let y = posY + sin(radians(angle)) * radius;
            textRotLayer(g, word, x, y, angle);
        }
    }
}

function textRotLayer(g, txt, x, y, angle) {
    g.push();
    g.translate(x, y);
    g.rotate(radians(angle));
    g.text(txt, 0, 0);
    g.pop();
}

//// NEW FUNCTION ////
function sinusSpacingLeft(g, w, h, bow, spread) {
    let target = ampEase * 40;
    let rowCount = 80
    let word = "&&&"
    spread = lerp(spread, target, ampEase > spread / 100 ? 0.1 : 0.03);
    g.textSize(20);
    g.textAlign(CENTER);
    g.noStroke();
    for (let i = 0; i < rowCount; i++) {
        let spac = abs(sin(frameCount * 0.04 + i * bow) * spread);
        textSpacingLayer(g, word.repeat(30), spac, 0, 20 * i);
    }
}
```

```
function textSpacingLayer(g, txt, spacing, x, y) {
    let totalWidth = 0;
    for (let char of txt) {
        totalWidth += g.textWidth(char) + spacing;
    }
    let posX = x;
    for (let char of txt) {
        g.text(char, posX, y);
        posX += g.textWidth(char) + spacing;
    }
}


//// NEW FUNCTION ////
function horizontalStretch(g, w, h, gridX, gridY, frameSpeed) {
    let gridWidth = w / gridX;
    let gridHeight = h / gridY;
    let letter = "@"
    g.textSize(gridHeight * 1.2);
    g.textAlign(LEFT, TOP);
    g.noStroke();

    // reseed random with a value that only changes every `speed` frames
    randomSeed(floor(frameCount / frameSpeed));

    for (let k = 0; k < gridY; k++) {
        let x = 0;
        for (let i = 0; i < gridX; i++) {
            let letterX = random(1) < 0.2 ? floor(random(3, 8)) : 1;
            let letterWidth = letterX * gridWidth;
            let scaleX = letterWidth / g.textWidth(letter);
            g.push();
            g.translate(x, k * gridHeight);
            g.scale(scaleX, 1);
            g.text(letter, 0, 0);
            g.pop();
            x += letterWidth;
        }
    }
```

```
}


//// NEW FUNCTION ////
function stretchedLetterGrid(g, w, h, amountX, amountY, frameSpeed) {
    let gridX = w / amountX;
    let gridY = h / amountY;
    let wordArray = ["*", "■", "#", "//"]
    let letterRep = [1, 2, 3, 4]
    g.textAlign(LEFT, TOP);
    g.noStroke();

    randomSeed(floor(frameCount / frameSpeed));

    for (let k = 0; k < amountY; k++) {
        for (let j = 0; j < amountX; j++) {
            g.textSize(gridY * 1.3);
            let i = (floor(frameCount / frameSpeed) + floor(random(4))) %
wordArray.length;
            let word = wordArray[i];
            let repeatedText = word.repeat(letterRep[i]);
            let scaleX = gridX / g.textWidth(repeatedText);
            g.push();
            g.translate(j * gridX, k * gridY);
            g.scale(scaleX, 1);
            g.text(repeatedText, 0, 0);
            g.pop();
        }
    }
}



//// NEW FUNCTION ////
function randomShapeGrid(g, w, h, numberX, numberY, frameSpeed) {
    g.noStroke();
    randomSeed(floor(frameCount / frameSpeed));
    for (let k = 0; k < numberY + 1; k++) {
        for (let i = 0; i < numberX + 1; i++) {
```

```
            shapeCell(g, i * w / numberX, k * h / numberY, w / numberX, h / 2,
frameSpeed);
        }
    }
}

function shapeCell(g, x, y, cellW, cellH, frameSpeed) {
    let gridY = 2;
    let gridX = 2;
    let columnWidth = cellW / gridX;
    let lineHeight = cellH / gridY;
    let shapeX1 = [1, 1, 1, 2, 1, 2];
    let shapeY1 = [2, 1, 2, 1, 1, 2];
    let i = floor(frameCount / frameSpeed) % shapeX1.length;
    let maxPosX = gridX - shapeX1[i];
    let maxPosY = gridY - shapeY1[i];
    let randomPosX = floor(random(0, max(1, maxPosX + 1)));
    let randomPosY = floor(random(0, max(1, maxPosY + 1)));
    let shapeWidth1 = shapeX1[i] * columnWidth;
    let shapeHeight1 = shapeY1[i] * lineHeight;
    g.rectMode(CORNER);
    g.rect(x + randomPosX * columnWidth, y + randomPosY * lineHeight,
shapeWidth1, shapeHeight1);
}


//// NEW FUNCTION ////
function letter4x4(g, w, h, repeatX, repeatY, frameSpeed) {
    let columnWidth = (w / repeatX) / 2;
    let lineHeight = (h / repeatY) / 2;
    let letters = ["[0]", "[=]", "[@]", "[?]"];
    let letterX = [[1,1,1,2,1,2], [1,1,1,0,1,0], [1,0,0,1,0,0], [0,2,0,1,1,0]];
    let letterY = [[2,1,2,1,1,2], [1,1,2,0,2,0], [1,0,0,1,0,0], [0,1,0,1,1,0]];
    let offsetX = [0, columnWidth, columnWidth, 0];
    let offsetY = [0, 0, lineHeight, lineHeight];
    g.textAlign(LEFT, TOP);
    let cellCount = 0;
    for (let ky = 0; ky < repeatY; ky++) {
        for (let kx = 0; kx < repeatX; kx++) {
            randomSeed(cellCount * 999);
```

```
            let randomOffset = floor(random(letterX[0].length));
            let i = (floor(frameCount / frameSpeed) + randomOffset) %
letterX[0].length;
            g.push();
            g.translate(kx * columnWidth * 2, ky * lineHeight * 2);
            for (let n = 0; n < 4; n++) {
                g.push();
                let letterWidth = letterX[n][i] * columnWidth;
                let letterHeight = letterY[n][i] * lineHeight;
                g.textSize(letterHeight);
                let scaleX = letterWidth / g.textWidth(letters[n]);
                g.scale(scaleX, 1);
                g.text(letters[n], offsetX[n] / scaleX, offsetY[n]);
                g.pop();
            }
            g.pop();
            cellCount++;
        }
    }
}
```



SATURDAY

- adding audioreactive components to the code snippets
- working on introduction: animation with blinking background
- turning additional coding snippets into functions and inserting them
- adding overlay full-screen text animations

CURRENT STATE OF THE CODING CANVAS

```
let leftLayer;
let centerLayer;
let rightLayer;

let bpm = 80
let blinkCount = (60 / bpm) * 60
let blinkLeft = 0
let blinkCenter = 0
let blinkRight = 0
let blinkMode = 1

// blinkMode = 1
// abwechselnd

// blinkRight / Left/ Center = 1 // blinkMode = 2
// alle Felder anderes Tempo

// blinkMode = 2 // rest 0
// stehender BG, s - w - s

function preload() {
  font = loadFont('/data/spacemono.ttf')
  //font = loadFont('/data/helvetica_neue.otf')
}

function setup() {
  createCanvas(windowWidth, windowHeight);
  leftLayer = createGraphics(width / 3, height);
  centerLayer = createGraphics(width / 3, height);
  rightLayer = createGraphics(width / 3, height);
  setupAudio(true) // global vars
```

```
  // a5.ease = .075 // set easing

  textFont(font)
}

function draw() {
  /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()
  let k = frameCount % blinkCount

  // background animation
  let k2 = frameCount % (blinkCount * 2) // left  - every 2 beats
  let k4 = frameCount % (blinkCount * 4) // right - every 4 beats
  let k8 = frameCount % (blinkCount * 8) // center - every 8 beats


  leftLayer.textFont(font)
  rightLayer.textFont(font)
  centerLayer.textFont(font)


  ///LEFT LAYER
  leftLayer.background(0);
  leftLayer.fill(255)

  //*blink animation
  if(blinkLeft == 1) {
    if(k2 < (blinkCount * 2) / 2) {
      leftLayer.background(255)
      leftLayer.fill(0)
    }
  }

  if(blinkMode == 1) {
    if(k < blinkCount / 2) {
      leftLayer.background(255)
      leftLayer.fill(0)
    }
  }
```

```
//*audioreactive - rhythm*
//sinusTwist(leftLayer, width / 3, height, 0.3, ampEase * 0.5)
crossedTrigo(leftLayer, width / 3, height, 0.5, 0.6 - ampEase * 0.02);
sinusSpacing(leftLayer, width / 3, height, 0.2, ampEase * 1.4);
//wordRotation(leftLayer, width / 3, height, 25, 8, 0.6);
//sinusSpacingLeft(leftLayer, width / 3, height, 0.1, 10);




//*flowy - ambient*
//sinusString(rightLayer, width / 3, height, 0.1, 80, 0.05);


//*frame by frame
//horizontalStretch(leftLayer, width / 3, height, 10, 5, 18, "@")
//stretchedLetterGrid(leftLayer, width/3, height, 8, 10, 18)
//randomShapeGrid(leftLayer, width / 3, height, 2, 8, 18);
//letter4x4(leftLayer, width/3, height, 2, 4, 18)

//



/// CENTER LAYER
centerLayer.background(255);
centerLayer.fill(0)



//*blink animation
if(blinkCenter == 1) {
  if(k8 < (blinkCount * 8) / 2) {
    centerLayer.background(0)
    centerLayer.fill(255)
  }
}

if(blinkMode == 1) {
  if(k < blinkCount / 2) {
```

```
    centerLayer.background(0)
    centerLayer.fill(255)
  }
}

if(blinkMode == 0) {
  centerLayer.background(0)
  centerLayer.fill(255)
}


//*audioreactive - rhythm*
sinusTwist(centerLayer, width / 3, height, 0.3, ampEase * 0.5)
//crossedTrigo(centerLayer, width / 3, height, 0.5, 0.6 - ampEase * 0.02);
//sinusSpacing(centerLayer, width / 3, height, 0.2, ampEase*1.4);
//wordRotation(centerLayer, width / 3, height, 25, 8, 0.6);
//sinusSpacingLeft(centerLayer, width / 3, height, 0.1, 10);




//*flowy - ambient*
//sinusString(centerLayer, width / 3, height, 0.1, 80, 0.05);


//*frame by frame
//horizontalStretch(centerLayer, width / 3, height, 10, 5, 18, "@")
//stretchedLetterGrid(centerLayer, width/3, height, 8, 10, 18)
//randomShapeGrid(centerLayer, width / 3, height, 2, 8, 18);
//letter4x4(centerLayer, width/3, height, 2, 4, 18)

//



// RIGHT LAYER
rightLayer.background(0);
rightLayer.fill(255)
```

```
//*blink animation
if(blinkRight == 1) {
  if(k4 < (blinkCount * 4) / 2) {
    rightLayer.background(255)
    rightLayer.fill(0)
  }
}


if(blinkMode == 1) {
  if(k < blinkCount / 2) {
    rightLayer.background(255)
    rightLayer.fill(0)
  }
}



//*audioreactive - rhythm*
//sinusTwist(rightLayer, width / 3, height, 0.3, ampEase * 0.5)
crossedTrigo(rightLayer, width / 3, height, 0.5, 0.6 - ampEase * 0.02);
sinusSpacing(rightLayer, width / 3, height, 0.2, ampEase * 1.4);
//wordRotation(rightLayer, width / 3, height, 25, 8, 0.6);
//sinusSpacingLeft(rightLayer, width / 3, height, 0.1, 10);




//*flowy - ambient*
//sinusString(rightLayer, width / 3, height, 0.1, 80, 0.05);


//*frame by frame
//horizontalStretch(rightLayer, width / 3, height, 10, 5, 18, "@")
//stretchedLetterGrid(rightLayer, width/3, height, 8, 10, 18)
//randomShapeGrid(rightLayer, width / 3, height, 2, 8, 18);
//letter4x4(rightLayer, width/3, height, 2, 4, 18)

//
```

```
  image(leftLayer, 0, 0);
  image(centerLayer, width / 3, 0);
  image(rightLayer, width / 3 * 2, 0);

  // DRAWING ON TOP
  fill(255)

  //textMorph("6", 600, 900, 1200, 18, 7)
  //glitchText("glitch", 350, 30)


}

//// NEW FUNCTION /////
function crossedTrigo(g, w, h, bow, shape) {
  const elementY = 100;
  var charset = "000000000---------000000000";
  g.noStroke();
  g.textAlign(CENTER, CENTER);
  g.textSize(h / elementY);
  for(let row = 0; row < elementY + 1; row++) {
    for(let col = 0; col < charset.length + 1; col++) {
      let posY = map(row, 0, elementY, 0, h);
      let magX = map(tan(radians(posY * bow + frameCount * 0.5)), -1, 1, -w *
shape, w * shape);
      let posX = map(col, 0, charset.length, -magX, magX);
      g.push();
      g.translate(w / 2 + posX, posY);
      g.text(charset[col], 0, 0);
      g.pop();
    }
  }
}


//// NEW FUNCTION /////
function sinusTwist(g, w, h, bow, tWidth) {
  let word = "*/*##*/*";
```

```
  g.noStroke();
  g.textSize(20);
  g.textAlign(CENTER);

  for(let i = 0; i < 220; i++) {
    let varSpacing = abs(sin(frameCount * 0.03 + i * bow) * tWidth);


    let totalWidth = 0;
    for(let char of word) {
      totalWidth += g.textWidth(char) + varSpacing * 10;
    }
    totalWidth -= varSpacing * 10;
    let posX = w / 2 - totalWidth / 2;
    for(let char of word) {
      g.text(char, posX, 5 * i);
      posX += g.textWidth(char) + varSpacing * 10;
    }
  }
}


//// NEW FUNCTION /////
function sinusString(g, w, h, bow, spread, speed) {
  let word = "!%!%!%!%";
  let letters = word.split('');
  g.noStroke();
  g.textSize(20);

  for(let j = 0; j < 50; j++) {
    for(let i = 0; i < letters.length; i++) {
      let offsetX = sin(frameCount * speed + j * bow) * spread * i;
      g.text(letters[i], w / 2 + offsetX * 0.5, j * 20);
    }
  }
}


//// NEW FUNCTION /////
function sinusSpacing(g, w, h, bow, spread) {
```

```
  let word = "[&*&]";
  g.noStroke();
  g.textSize(20);
  g.textAlign(LEFT);

  for(let i = 0; i < 70; i++) {
    let spac = abs(sin(frameCount * 0.03 + i * bow) * spread);

    let totalWidth = 0;
    for(let char of word.repeat(8)) {
      totalWidth += g.textWidth(char) + spac;
    }
    totalWidth -= spac;
    let posX = w / 2 - totalWidth / 2;
    for(let char of word.repeat(30)) {
      g.text(char, posX, 20 * i);
      posX += g.textWidth(char) + spac;
    }
  }
}


//// NEW FUNCTION /////
function wordRotation(g, w, h, wordCount, rowCount, speed) {
  let word = "-#-%-#-";
  g.textSize(22);
  g.noStroke();
  let posX = w / 2;
  let posY = h / 2;
  let wordWidth = g.textWidth(word);
  for(let j = 0; j < rowCount; j++) {
    let radiusVar = sin(frameCount * 0.2) * 3 + 5 * ampEase;
    let radius = radiusVar + 5 + j * (wordWidth + 10);
    let direction = (j % 2 === 0) ? 1 : -1;
    let rotation = 360 / wordCount;
    for(let i = 0; i < wordCount; i++) {
      let rowOffset = frameCount * speed * direction;
      let angle = i * rotation + rowOffset;
      let x = posX + cos(radians(angle)) * radius;
      let y = posY + sin(radians(angle)) * radius;
```

```
      textRotLayer(g, word, x, y, angle);
    }
  }
}

function textRotLayer(g, txt, x, y, angle) {
  g.push();
  g.translate(x, y);
  g.rotate(radians(angle));
  g.text(txt, 0, 0);
  g.pop();
}

//// NEW FUNCTION ////
function sinusSpacingLeft(g, w, h, bow, spread) {
  let target = fftEase[50] * 10;
  let rowCount = 80
  let word = "&&&"
  spread = lerp(spread, target, ampEase > spread / 100 ? 0.1 : 0.03);
  g.textSize(20);
  g.textAlign(CENTER);
  g.noStroke();
  for(let i = 0; i < rowCount; i++) {
    let spac = abs(sin(frameCount * 0.04 + i * bow) * spread);
    textSpacingLayer(g, word.repeat(30), spac, 0, 20 * i);
  }
}

function textSpacingLayer(g, txt, spacing, x, y) {
  let totalWidth = 0;
  for(let char of txt) {
    totalWidth += g.textWidth(char) + spacing;
  }
  let posX = x;
  for(let char of txt) {
    g.text(char, posX, y);
    posX += g.textWidth(char) + spacing;
  }
}
```

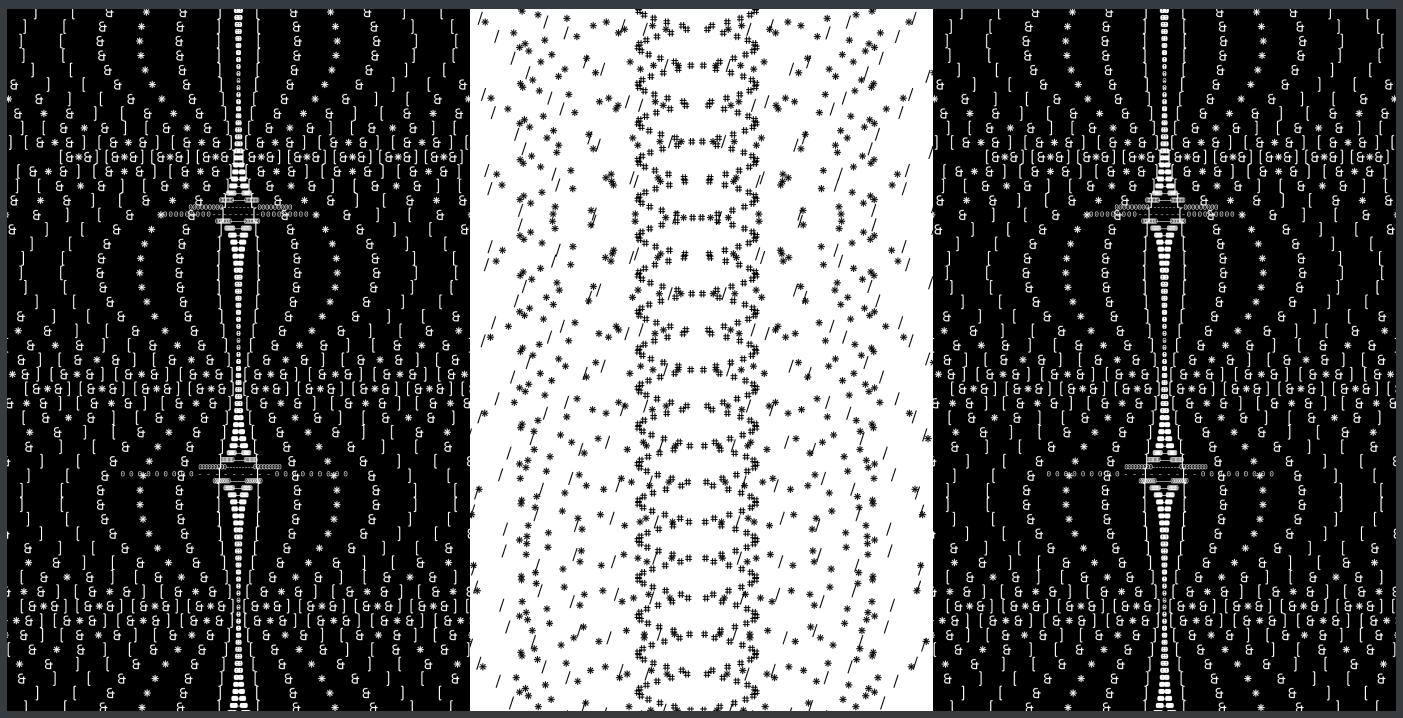
```
//// NEW FUNCTION ////
function horizontalStretch(g, w, h, gridX, gridY, frameSpeed, letter) {
  let gridWidth = w / gridX;
  let gridHeight = h / gridY;
  g.textSize(gridHeight * 1.2);
  g.textAlign(LEFT, TOP);
  g.noStroke();

  // reseed random with a value that only changes every `speed` frames
  randomSeed(floor(frameCount / frameSpeed));

  for(let k = 0; k < gridY; k++) {
    let x = 0;
    for(let i = 0; i < gridX; i++) {
      let letterX = random(1) < 0.2 ? floor(random(3, 8)) : 1;
      let letterWidth = letterX * gridWidth;
      let scaleX = letterWidth / g.textWidth(letter);
      g.push();
      g.translate(x, k * gridHeight);
      g.scale(scaleX, 1);
      g.text(letter, 0, 0);
      g.pop();
      x += letterWidth;
    }
  }
}


//// NEW FUNCTION ////
function stretchedLetterGrid(g, w, h, amountX, amountY, frameSpeed) {
  let gridX = w / amountX;
  let gridY = h / amountY;
  let wordArray = ["*", "■", "#", "/"]
  let letterRep = [1, 2, 3, 4]
  g.textAlign(LEFT, TOP);
  g.noStroke();

  randomSeed(floor(frameCount / frameSpeed));
```

```
  for(let k = 0; k < amountY; k++) {
    for(let j = 0; j < amountX; j++) {
      g.textSize(gridY * 1.3);
      let i = (floor(frameCount / frameSpeed) + floor(random(4))) %
wordArray.length;
      let word = wordArray[i];
      let repeatedText = word.repeat(letterRep[i]);
      let scaleX = gridX / g.textWidth(repeatedText);
      g.push();
      g.translate(j * gridX, k * gridY);
      g.scale(scaleX, 1);
      g.text(repeatedText, 0, 0);
      g.pop();
    }
  }
}



//// NEW FUNCTION ////
function randomShapeGrid(g, w, h, numberX, numberY, frameSpeed) {
  g.noStroke();
  randomSeed(floor(frameCount / frameSpeed));
  for(let k = 0; k < numberY + 1; k++) {
    for(let i = 0; i < numberX + 1; i++) {
      shapeCell(g, i * w / numberX, k * h / numberY, w / numberX, h / 2,
frameSpeed);
    }
  }
}

function shapeCell(g, x, y, cellW, cellH, frameSpeed) {
  let gridY = 2;
  let gridX = 2;
  let columnWidth = cellW / gridX;
  let lineHeight = cellH / gridY;
  let shapeX1 = [1, 1, 1, 2, 1, 2];
  let shapeY1 = [2, 1, 2, 1, 1, 2];
  let i = floor(frameCount / frameSpeed) % shapeX1.length;
  let maxPosX = gridX - shapeX1[i];
```

```
  let maxPosY = gridY - shapeY1[i];
  let randomPosX = floor(random(0, max(1, maxPosX + 1)));
  let randomPosY = floor(random(0, max(1, maxPosY + 1)));
  let shapeWidth1 = shapeX1[i] * columnWidth;
  let shapeHeight1 = shapeY1[i] * lineHeight;
  g.rectMode(CORNER);
  g.rect(x + randomPosX * columnWidth, y + randomPosY * lineHeight, shapeWidth1,
shapeHeight1);
}


//// NEW FUNCTION ////
function letter4x4(g, w, h, repeatX, repeatY, frameSpeed) {
  let columnWidth = (w / repeatX) / 2;
  let lineHeight = (h / repeatY) / 2;
  let letters = ["[0]", "[=]", "[@]", "[?]"];
  let letterX = [
    [1, 1, 1, 2, 1, 2],
    [1, 1, 1, 0, 1, 0],
    [1, 0, 0, 1, 0, 0],
    [0, 2, 0, 1, 1, 0]
  ];
  let letterY = [
    [2, 1, 2, 1, 1, 2],
    [1, 1, 2, 0, 2, 0],
    [1, 0, 0, 1, 0, 0],
    [0, 1, 0, 1, 1, 0]
  ];
  let offsetX = [0, columnWidth, columnWidth, 0];
  let offsetY = [0, 0, lineHeight, lineHeight];
  g.textAlign(LEFT, TOP);
  let cellCount = 0;
  for(let ky = 0; ky < repeatY; ky++) {
    for(let kx = 0; kx < repeatX; kx++) {
      randomSeed(cellCount * 999);
      let randomOffset = floor(random(letterX[0].length));
      let i = (floor(frameCount / frameSpeed) + randomOffset) %
letterX[0].length;
      g.push();
      g.translate(kx * columnWidth * 2, ky * lineHeight * 2);
```

```
      for(let n = 0; n < 4; n++) {
        g.push();
        let letterWidth = letterX[n][i] * columnWidth;
        let letterHeight = letterY[n][i] * lineHeight;
        g.textSize(letterHeight);
        let scaleX = letterWidth / g.textWidth(letters[n]);
        g.scale(scaleX, 1);
        g.text(letters[n], offsetX[n] / scaleX, offsetY[n]);
        g.pop();
      }
      g.pop();
      cellCount++;
    }
  }
}


function textMorph(letter, x, y, size, offsetRange, frameSpeed) {
  let pts = font.textToPoints(letter, x, y, size, {
    sampleFactor: 1,
    simplifyThreshold: 0
  })

  randomSeed(floor(frameCount / frameSpeed))
  let offsetX = random(-offsetRange, offsetRange)
  let offsetY = random(-offsetRange, offsetRange)
  let sign = floor(random(2)) * 2 - 1 // randomly -1 or 1

  noStroke()
  beginShape()
  for(let i = 0; i < pts.length; i++) {
    let ox = sign * (i + 1) * 0.03 * offsetX
    let oy = sign * (i + 1) * 0.003 * offsetY
    vertex(pts[i].x + ox, pts[i].y + oy)
  }
  endShape(CLOSE)
}

function glitchText(txt, size, glitchRange) {
  textAlign(CENTER, CENTER)
```

```
    let x = width / 2
    let y = height / 2 - size / 3
    let glitch = glitchRange
    textSize(size)

    fill(0, 255, 0)
    text(txt, x + random(-glitch, glitch), y + random(-glitch, glitch))

    fill(255, 0, 255)
    text(txt, x + random(-glitch, glitch), y + glitch)

    fill(255)
    text(txt, x + random(-glitch, glitch), y + random(-glitch, glitch))
  }
```



SUNDAY

- adding new code snippets as functions to the canvas
- fine-tuning the audioreactive parts with the music
- trying out "co-coding"

- planning the performance

FINAL CODING CANVAS

```
let leftLayer;
let centerLayer;
let rightLayer;

let bpm = 145
let blinkCount = (60 / bpm) * 60
let blinkLeft = 0
let blinkCenter = 0
let blinkRight = 0
let blinkMode = 2

// blinkMode = 1
// abwechselnd

// blinkRight / Left / Center = 1 // blinkMode = 2
// alle Felder anderes Tempo

// blinkMode = 2 // rest 0
// stehender BG, s - w - s


function setup() {
  createCanvas(windowWidth, windowHeight);
  leftLayer = createGraphics(width / 3, height);
  centerLayer = createGraphics(width / 3, height);
  rightLayer = createGraphics(width / 3, height);
  setupAudio(true) // global vars
  // a5.ease = .075 // set easing

  //textFont(font)
}

function draw() {
  /* audio vars: amp, ampEase, fft, fftEase, waveform, waveformEase */
  updateAudio()
```

```
let k = frameCount % blinkCount

// background animation
let k2 = frameCount % (blinkCount * 2) // left  - every 2 beats
let k4 = frameCount % (blinkCount * 4) // right - every 4 beats
let k8 = frameCount % (blinkCount * 8) // center - ev ery 8 beats


leftLayer.textFont('Helvetica')
rightLayer.textFont('Helvetica')
centerLayer.textFont('Helvetica')

textSize(width / 5)
textAlign(CENTER, CENTER)




/// CENTER LAYER
centerLayer.background(255);
centerLayer.fill(0)

//*blink animation
if(blinkCenter == 1) {
  if(k8 < (blinkCount * 8) / 2) {
    centerLayer.background(0)
    centerLayer.fill(255)
  }
}

if(blinkMode == 1) {
  if(k < blinkCount / 2) {
    centerLayer.background(0)
    centerLayer.fill(255)
  }
}

if(blinkMode == 0) {
  centerLayer.background(0)
  centerLayer.fill(255)
```

```
}


//*audioreactive - rhythm*
//sinusTwist(centerLayer, width / 3, height, 0.3, fftEase[10] * 0.025)
//crossedTrigo(centerLayer, width / 3, height, 0.5, 0.6 - ampEase*0.004);
//sinusSpacing(centerLayer, width / 3, height, 0.2, fftEase[80]*0.05);
//wordRotation(centerLayer, width / 3, height, 25, 8, 0.6);
//sinusSpacingLeft(centerLayer, width / 3, height, 0.1, 10);




//*flowy - ambient*
//sinusString(centerLayer, width / 3, height, 0.1, 80, 0.05);

vertLetterStretch(centerLayer, width/3, height, "OVER")
//vertLetterStretchAlt(centerLayer, width / 3, height, "DROWN")


//*frame by frame
// horizontalStretch(centerLayer, width / 3, height, 10, 5, 18, "@")
// stretchedLetterGrid(centerLayer, width/3, height, 8, 10, 18)
// randomShapeGrid(centerLayer, width / 3, height, 2, 8, 18);
//letter4x4(centerLayer, width/3, height, 2, 4, 18)

//


/////////----------------------------


///LEFT LAYER
leftLayer.background(0);
leftLayer.fill(255)

//*blink animation
if(blinkLeft == 1) {
  if(k2 < (blinkCount * 2) / 2) {
    leftLayer.background(255)
```

```
    leftLayer.fill(0)
  }
}

if(blinkMode == 1) {
  if(k < blinkCount / 2) {
    leftLayer.background(255)
    leftLayer.fill(0)
  }
}

//*audioreactive - rhythm*
//sinusTwist(leftLayer, width / 3, height, 0.3, fftEase[10] * 0.025)
crossedTrigo(leftLayer, width / 3, height, 0.4, ampEase * 0.04);
//sinusSpacing(leftLayer, width / 3, height, 0.2, fftEase[80] * 0.4);
//wordRotation(leftLayer, width / 3, height, 25, 8, 0.6);
// sinusSpacingLeft(leftLayer, width / 3, height, 0.1, 10);




//*flowy - ambient*
// sinusString(leftLayer, width / 3, height, 0.1, 80, 0.05);

// vertLetterStretch(leftLayer, width/3, height, "OVER")
// vertLetterStretchAlt(leftLayer, width / 3, height, "OVER")


//*frame by frame
 horizontalStretch(leftLayer, width / 3, height, 10, 5, 18, "@")
// stretchedLetterGrid(leftLayer, width / 3, height, 8, 10, 18)
// randomShapeGrid(leftLayer, width / 3, height, 2, 8, 18);
//letter4x4(leftLayer, width/3, height, 2, 4, 18)

//




// RIGHT LAYER
```

```
rightLayer.background(0);
rightLayer.fill(255)



//*blink animation
if(blinkRight == 1) {
  if(k4 < (blinkCount * 4) / 2) {
    rightLayer.background(255)
    rightLayer.fill(0)
  }
}


if(blinkMode == 1) {
  if(k < blinkCount / 2) {
    rightLayer.background(255)
    rightLayer.fill(0)
  }
}



//*audioreactive - rhythm*
// sinusTwist(rightLayer, width / 3, height, 0.3, fftEase[10] * 0.025)
// crossedTrigo(rightLayer, width / 3, height, 0.5, ampEase * 0.04);
// sinusSpacing(rightLayer, width / 3, height, 0.2, fftEase[80] * 0.05);
wordRotation(rightLayer, width / 3, height, 25, 8, 0.6);
// sinusSpacingLeft(rightLayer, width / 3, height, 0.1, 10);




//*flowy - ambient*
// sinusString(rightLayer, width / 3, height, 0.1, 80, 0.05);

/// /vertLetterStretch(rightLayer, width/3, height, "LOAD")
// vertLetterStretchAlt(rightLayer, width / 3, height, "OVER")
```

```
  //*frame by frame
  // horizontalStretch(rightLayer, width / 3, height, 10, 5, 18, "@")
  stretchedLetterGrid(rightLayer, width/3, height, 8, 10, 18)
  // randomShapeGrid(rightLayer, width / 3, height, 2, 8, 18);
  // letter4x4(rightLayer, width/3, height, 2, 4, 18)

  //


  image(leftLayer, 0, 0);
  image(centerLayer, width / 3, 0);
  image(rightLayer, width / 3 * 2, 0);



  // DRAWING ON TOP
  fill(255)

  //glitchy("stimulate")

}




//// NEW FUNCTION /////
function crossedTrigo(g, w, h, bow, shape) {
  const elementY = 80;
  var charset = "000000000---------000000000";
  g.noStroke();
  g.textAlign(CENTER, CENTER);
  g.textSize(h / elementY);
  for(let row = 0; row < elementY + 1; row++) {
    for(let col = 0; col < charset.length + 1; col++) {
      let posY = map(row, 0, elementY, 0, h);
      let magX = map(tan(radians(posY * bow + frameCount * 0.5)), -1, 1, -w *
shape, w * shape);
      let posX = map(col, 0, charset.length, -magX, magX);
      g.push();
```

```
      g.translate(w / 2 + posX, posY);
      g.text(charset[col], 0, 0);
      g.pop();
    }
  }
}


//// NEW FUNCTION /////
function sinusTwist(g, w, h, bow, tWidth) {
  let word = "*/*##*/*";
  g.noStroke();
  g.textSize(20);
  g.textAlign(CENTER);

  for(let i = 0; i < 220; i++) {
    let varSpacing = abs(sin(frameCount * 0.03 + i * bow) * tWidth);


    let totalWidth = 0;
    for(let char of word) {
      totalWidth += g.textWidth(char) + varSpacing * 10;
    }
    totalWidth -= varSpacing * 10;
    let posX = w / 2 - totalWidth / 2;
    for(let char of word) {
      g.text(char, posX, 5 * i);
      posX += g.textWidth(char) + varSpacing * 10;
    }
  }
}


//// NEW FUNCTION /////
function sinusString(g, w, h, bow, spread, speed) {
  let word = "!%!%!%!%";
  let letters = word.split('');
  g.noStroke();
  g.textSize(20);
```

```
  for(let j = 0; j < 52; j++) {
    for(let i = 0; i < letters.length; i++) {
      let offsetX = sin(frameCount * speed + j * bow) * spread * i;
      g.text(letters[i], w / 2 + offsetX * 0.5, j * 20);
    }
  }
}


//// NEW FUNCTION /////
function sinusSpacing(g, w, h, bow, spread) {
  let word = "[&*&]";
  g.noStroke();
  g.textSize(20);
  g.textAlign(LEFT);

  for(let i = 0; i < 70; i++) {
    let spac = abs(sin(frameCount * 0.03 + i * bow) * spread);

    let totalWidth = 0;
    for(let char of word.repeat(8)) {
      totalWidth += g.textWidth(char) + spac;
    }
    totalWidth -= spac;
    let posX = w / 2 - totalWidth / 2;
    for(let char of word.repeat(10)) {
      g.text(char, posX, 20 * i);
      posX += g.textWidth(char) + spac;
    }
  }
}


//// NEW FUNCTION /////
function wordRotation(g, w, h, wordCount, rowCount, speed) {
  let word = "-#-%-#-";
  g.textSize(22);
  g.noStroke();
  let posX = w / 2;
  let posY = h / 2;
```

```
  let wordWidth = g.textWidth(word);
  for(let j = 0; j < rowCount; j++) {
    let radiusVar = sin(frameCount * 0.2) * 3 + 0.4 * ampEase;
    let radius = radiusVar + 5 + j * (wordWidth + 10);
    let direction = (j % 2 === 0) ? 1 : -1;
    let rotation = 360 / wordCount;
    for(let i = 0; i < wordCount; i++) {
      let rowOffset = frameCount * speed * direction;
      let angle = i * rotation + rowOffset;
      let x = posX + cos(radians(angle)) * radius;
      let y = posY + sin(radians(angle)) * radius;
      textRotLayer(g, word, x, y, angle);
    }
  }
}

function textRotLayer(g, txt, x, y, angle) {
  g.push();
  g.translate(x, y);
  g.rotate(radians(angle));
  g.text(txt, 0, 0);
  g.pop();
}

//// NEW FUNCTION ////
function sinusSpacingLeft(g, w, h, bow, spread) {
  let target = fftEase[50] * 4;
  let rowCount = 80
  let word = "&&&"
  spread = lerp(spread, target, ampEase > spread / 100 ? 0.1 : 0.03);
  g.textSize(20);
  g.textAlign(CENTER);
  g.noStroke();
  for(let i = 0; i < rowCount; i++) {
    let spac = abs(sin(frameCount * 0.04 + i * bow) * spread);
    textSpacingLayer(g, word.repeat(30), spac, 0, 20 * i);
  }
}

function textSpacingLayer(g, txt, spacing, x, y) {
```

```
  let totalWidth = 0;
  for(let char of txt) {
    totalWidth += g.textWidth(char) + spacing;
  }
  let posX = x;
  for(let char of txt) {
    g.text(char, posX, y);
    posX += g.textWidth(char) + spacing;
  }
}


//// NEW FUNCTION ////
function horizontalStretch(g, w, h, gridX, gridY, frameSpeed, letter) {
  let gridWidth = w / gridX;
  let gridHeight = h / gridY;
  g.textSize(gridHeight * 1.2);
  g.textAlign(LEFT, TOP);
  g.noStroke();

  // reseed random with a value that only changes every `speed` frames
  randomSeed(floor(frameCount / frameSpeed));

  for(let k = 0; k < gridY; k++) {
    let x = 0;
    for(let i = 0; i < gridX; i++) {
      let letterX = random(1) < 0.2 ? floor(random(3, 8)) : 1;
      let letterWidth = letterX * gridWidth;
      let scaleX = letterWidth / g.textWidth(letter);
      g.push();
      g.translate(x, k * gridHeight);
      g.scale(scaleX, 1);
      g.text(letter, 0, 0);
      g.pop();
      x += letterWidth;
    }
  }
}
```

```
//// NEW FUNCTION ////
function stretchedLetterGrid(g, w, h, amountX, amountY, frameSpeed) {
  let gridX = w / amountX;
  let gridY = h / amountY;
  let wordArray = ["*", "■", "#", "/"]
  let letterRep = [1, 2, 3, 4]
  g.textAlign(LEFT, TOP);
  g.noStroke();

  randomSeed(floor(frameCount / frameSpeed));

  for(let k = 0; k < amountY; k++) {
    for(let j = 0; j < amountX; j++) {
      g.textSize(gridY * 1.3);
      let i = (floor(frameCount / frameSpeed) + floor(random(4))) %
wordArray.length;
      let word = wordArray[i];
      let repeatedText = word.repeat(letterRep[i]);
      let scaleX = gridX / g.textWidth(repeatedText);
      g.push();
      g.translate(j * gridX, k * gridY);
      g.scale(scaleX, 1);
      g.text(repeatedText, 0, 0);
      g.pop();
    }
  }
}



//// NEW FUNCTION ////
function randomShapeGrid(g, w, h, numberX, numberY, frameSpeed) {
  g.noStroke();
  randomSeed(floor(frameCount / frameSpeed));
  for(let k = 0; k < numberY + 1; k++) {
    for(let i = 0; i < numberX + 1; i++) {
      shapeCell(g, i * w / numberX, k * h / numberY, w / numberX, h / 2,
frameSpeed);
    }
  }
```

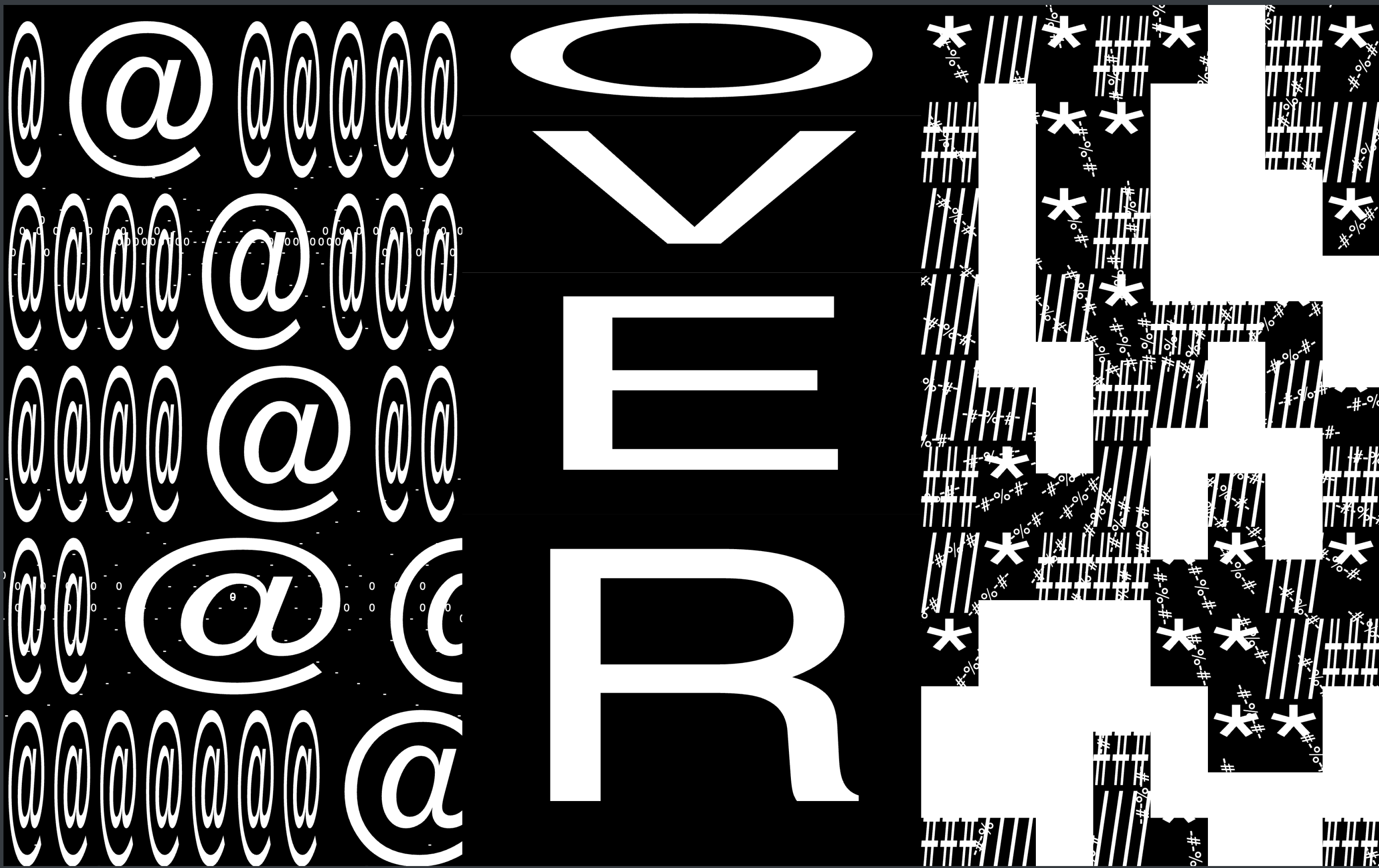
```
}

function shapeCell(g, x, y, cellW, cellH, frameSpeed) {
  let gridY = 2;
  let gridX = 2;
  let columnWidth = cellW / gridX;
  let lineHeight = cellH / gridY;
  let shapeX1 = [1, 1, 1, 2, 1, 2];
  let shapeY1 = [2, 1, 2, 1, 1, 2];
  let i = floor(frameCount / frameSpeed) % shapeX1.length;
  let maxPosX = gridX - shapeX1[i];
  let maxPosY = gridY - shapeY1[i];
  let randomPosX = floor(random(0, max(1, maxPosX + 1)));
  let randomPosY = floor(random(0, max(1, maxPosY + 1)));
  let shapeWidth1 = shapeX1[i] * columnWidth;
  let shapeHeight1 = shapeY1[i] * lineHeight;
  g.rectMode(CORNER);
  g.rect(x + randomPosX * columnWidth, y + randomPosY * lineHeight, shapeWidth1,
shapeHeight1);
}


//// NEW FUNCTION ////
function letter4x4(g, w, h, repeatX, repeatY, frameSpeed) {
  let columnWidth = (w / repeatX) / 2;
  let lineHeight = (h / repeatY) / 2;
  let letters = ["[0]", "[=]", "[@]", "[?]"];
  let letterX = [
    [1, 1, 1, 2, 1, 2],
    [1, 1, 1, 0, 1, 0],
    [1, 0, 0, 1, 0, 0],
    [0, 2, 0, 1, 1, 0]
  ];
  let letterY = [
    [2, 1, 2, 1, 1, 2],
    [1, 1, 2, 0, 2, 0],
    [1, 0, 0, 1, 0, 0],
    [0, 1, 0, 1, 1, 0]
  ];
  let offsetX = [0, columnWidth, columnWidth, 0];
```

```
  let offsetY = [0, 0, lineHeight, lineHeight];
  g.textAlign(LEFT, TOP);
  let cellCount = 0;
  for(let ky = 0; ky < repeatY; ky++) {
    for(let kx = 0; kx < repeatX; kx++) {
      randomSeed(cellCount * 999);
      let randomOffset = floor(random(letterX[0].length));
      let i = (floor(frameCount / frameSpeed) + randomOffset) %
letterX[0].length;
      g.push();
      g.translate(kx * columnWidth * 2, ky * lineHeight * 2);
      for(let n = 0; n < 4; n++) {
        g.push();
        let letterWidth = letterX[n][i] * columnWidth;
        let letterHeight = letterY[n][i] * lineHeight;
        g.textSize(letterHeight);
        let scaleX = letterWidth / g.textWidth(letters[n]);
        g.scale(scaleX, 1);
        g.text(letters[n], offsetX[n] / scaleX, offsetY[n]);
        g.pop();
      }
      g.pop();
      cellCount++;
    }
  }
}


function textMorph(letter, x, y, size, offsetRange, frameSpeed) {
  let pts = font.textToPoints(letter, x, y, size, {
    sampleFactor: 1,
    simplifyThreshold: 0
  })

  randomSeed(floor(frameCount / frameSpeed))
  let offsetX = random(-offsetRange, offsetRange)
  let offsetY = random(-offsetRange, offsetRange)
  let sign = floor(random(2)) * 2 - 1 // randomly -1 or 1

  noStroke()
```

```
  beginShape()
  for(let i = 0; i < pts.length; i++) {
    let ox = sign * (i + 1) * 0.03 * offsetX
    let oy = sign * (i + 1) * 0.003 * offsetY
    vertex(pts[i].x + ox, pts[i].y + oy)
  }
  endShape(CLOSE)
}

function glitchy(txt) {
  let glitch = fftEase[70] * 0.04;

  //Dunkel
  fill(80);
  text(
    txt,
    width / 2 + random(-glitch, glitch),
    height / 2 + random(-glitch, glitch)
  );

  //Mittel
  fill(0, 0, 255);
  text(
    txt,
    width / 2 + random(-glitch, glitch),
    height / 2 + glitch
  );

  // Haupttext
  fill(255);
  text(
    txt,
    width / 2 + random(-glitch, glitch),
    height / 2 + random(-glitch, glitch)
  );
}



function vertLetterStretch(g, w, h, word) {
```

```
    let letters = word.split('');
    let num = letters.length;
    let t = millis() * 0.001;
    let freq = (bpm / 60) / 4;

    let weights = [];
    let total = 0;
    for(let i = 0; i < num; i++) {
      let wave = sin(TWO_PI * freq * t + i * 0.4);
      let wt = map(wave, -1, 1, 0.3, 2);
      weights[i] = wt;
      total += wt;
    }

    let y = 0;
    let wave = sin(TWO_PI * freq * t * 2);
    for(let i = 0; i < num; i++) {
      let boxH = (weights[i] / total) * h;
      g.push();
      g.translate(w / 2, y + boxH / 2);

      let bg = wave > 0 ? 255 : 0;
      g.fill(bg);
      g.noStroke();
      g.rectMode(CENTER);
      g.rect(0, 0, w, boxH);

      let fg = wave > 0 ? 0 : 255;
      g.fill(fg);
      g.textAlign(CENTER, CENTER);
      g.textSize(650);
      g.scale(1, boxH / 650);
      g.text(letters[i % num], 0, 0);
      g.pop();

      y += boxH;
    }
  }
```

```
function vertLetterStretchAlt(g, w, h, word) {
  let letters = word.split('');
  let num = letters.length;
  let t = millis() * 0.001;
  let freq = (bpm / 60) / 4;

  let weights = [];
  let total = 0;
  for(let i = 0; i < num; i++) {
    let wave = sin(TWO_PI * freq * t + i * 0.4);
    let wt = map(wave, -1, 1, 0.3, 2);
    weights[i] = wt;
    total += wt;
  }

  let y = 0;
  for(let i = 0; i < num; i++) {
    let boxH = (weights[i] / total) * h;
    g.push();
    g.translate(w / 2, y + boxH / 2);

    let col = (i % 2 === 0) ? 255 : 0;
    g.fill(col);
    g.noStroke();
    g.rectMode(CENTER);
    g.rect(0, 0, w, boxH);

    g.fill(col === 255 ? 0 : 255);
    g.textAlign(CENTER, CENTER);
    g.textSize(650);
    g.scale(1, boxH / 650);
    g.text(letters[i % num], 0, 0);
    g.pop();

    y += boxH;
  }
} //
```


OVER